Storage Techniques Mini-Project: Implementing a Storage Platform with TrueNAS

Oct 17, 2026 · @islem · ESPRIT School of Engineering, 5ArcTIC10

1. Introduction

This report documents the design, deployment and testing of a complete network storage platform built on TrueNAS Community Edition 25.10.7. A single TrueNAS server exposes the same pool of disks in three different ways: as a file share for Linux (NFS), as a file share for Windows (SMB), and as a raw network disk (iSCSI). Every step was carried out on a virtual lab and is backed by screenshots and command output.

1.1 Context

Modern information systems separate compute from storage. Instead of keeping data on each server's local disk, organisations centralise it on dedicated storage systems: a NAS (Network Attached Storage) that shares files, or a SAN (Storage Area Network) that shares disks. TrueNAS is an open-source storage operating system that can play both roles. It is built on Linux (Debian) and on ZFS, a combined file system and volume manager that handles RAID, integrity checking, compression and snapshots by itself.

1.2 Objectives

The mini-project asks to:

1. Prepare a TrueNAS server and identify its raw disks.
2. Group the disks into VDEVs and pools using different RAID levels, and compare usable capacity and fault tolerance.
3. Configure an NFS share and use it from a Linux client.
4. Configure an SMB share and use it from a Windows client.
5. Configure an iSCSI target and connect a Windows initiator to it.
6. Capture and analyse the iSCSI traffic with Wireshark, including the TCP connection, the Initiator/Target login, and READ/WRITE operations.
7. Compare file-level and block-level storage and answer the synthesis questions.

1.3 Results at a glance

Part	Result
TrueNAS server	Installed on VMware Workstation, reachable at 192.168.64.149
Storage pools	3 pools built on 7 data disks: Mirror (NFS), RAIDZ1 (SMB), Mirror (iSCSI), all ONLINE
NFS	Share mounted on Ubuntu with NFSv4.2; create, modify, copy and delete verified on the server
SMB	Share opened from Windows Explorer as `smbuser` with SMB authentication; read and write verified on the server
iSCSI	5 GiB zvol presented as LUN 0, formatted NTFS by Windows and used as drive `I:`
Wireshark	65,450 packets captured; TCP handshake, iSCSI login, Read(10) and Write(10) identified and explained

2. Lab environment and tools

The whole lab runs on one Windows laptop with VMware Workstation. The laptop itself plays the role of the Windows client, so only two virtual machines are needed.

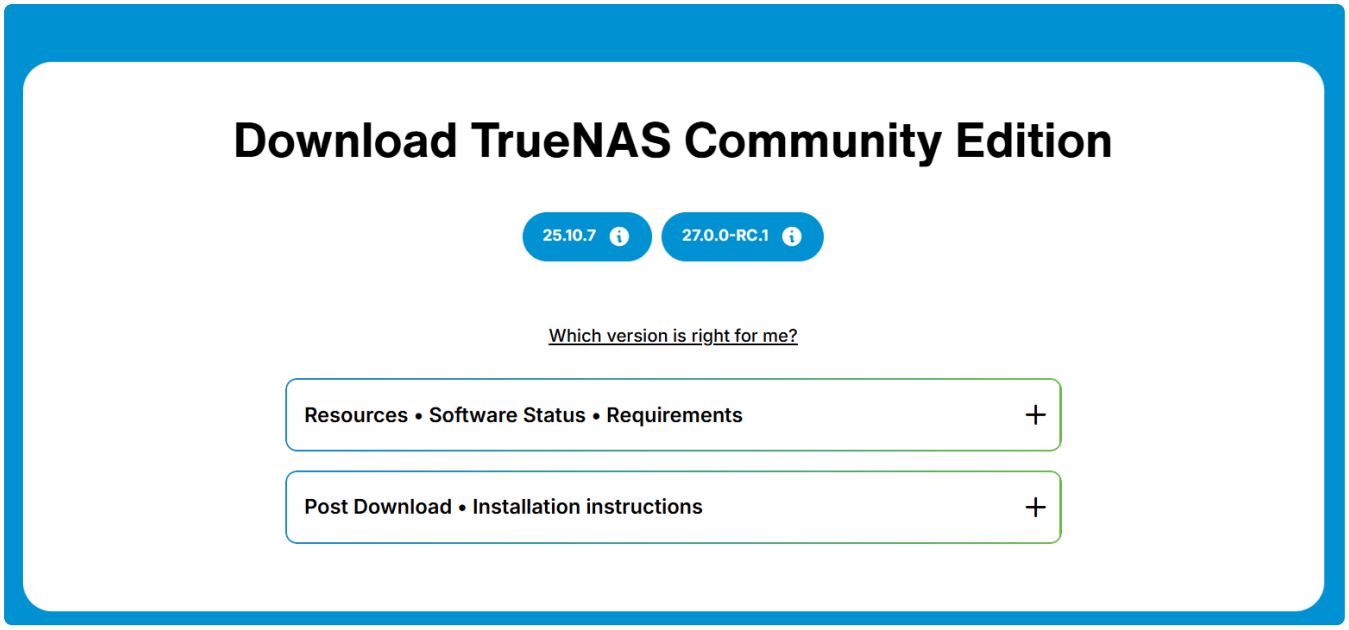
2.1 Machines

Machine	Role	Resources	IP address
TrueNAS VM	Storage server (NAS + SAN)	2 vCPU, 8 GB RAM, 1 × 20 GB boot disk + 7 × 10 GB data disks	192.168.64.149
Ubuntu 64-bit VM	NFS client	Existing VM, NAT network	192.168.64.147
Windows host (ASUS laptop)	SMB client, iSCSI initiator, Wireshark	Physical machine	192.168.64.1 (VMnet8 adapter)

2.2 Software and tools

Tool	Version	Why it was used
VMware Workstation	25H2	Hypervisor that runs the TrueNAS and Ubuntu VMs and provides the NAT virtual network VMnet8
TrueNAS Community Edition	25.10.7 "Goldeye" (stable)	Storage operating system: ZFS pools, NFS, SMB and iSCSI services, web UI
ZFS	Bundled with TrueNAS	Combined volume manager and file system: VDEVs, pools, datasets, zvols, LZ4 compression
Ubuntu + `nfs-common`	Ubuntu 24.04 (noble), nfs-common 2.6.4	NFS client tools (`mount.nfs` , `showmount`) and `rpcbind`
Windows File Explorer	Windows 11	Built-in SMB client used to open `\\192.168.64.149\smb-share`
Microsoft iSCSI Initiator	Built into Windows	Software initiator that logs in to the iSCSI target
Disk Management (`diskmgmt.msc`)	Built into Windows	Initialises, partitions and formats the iSCSI LUN
Wireshark + Npcap	Wireshark 4.6.9, Npcap 1.88	Packet capture and protocol dissection of the iSCSI traffic
TrueNAS Shell / `midclt`	Bundled	Command line used to verify files on the server and to create pools through the TrueNAS API

The TrueNAS version was chosen deliberately: the download page offered 25.10.7 (stable) and 27.0.0-RC.1 (release candidate). A release candidate is still under test, so the stable release was selected for reliability.

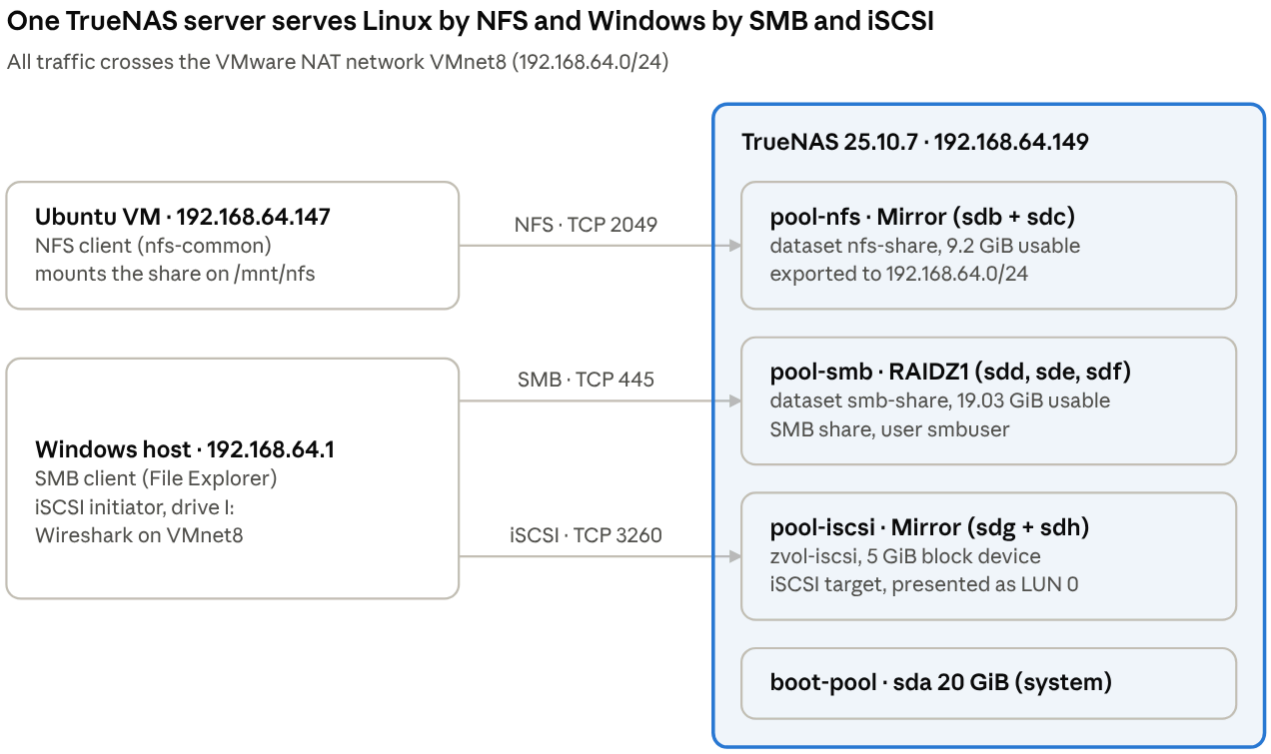


2.3 Network

All machines are attached to VMware's NAT network VMnet8 (192.168.64.0/24). VMware's DHCP server gives each VM an address, and the Windows host reaches the VMs through its own virtual adapter "VMware Network Adapter VMnet8" (192.168.64.1). This is also the interface Wireshark listens on, so all storage traffic between Windows and TrueNAS can be captured.

3. Lab architecture

One TrueNAS server holds eight virtual disks. The first is reserved for the operating system; the other seven are grouped into three ZFS pools, one per protocol. Each pool is reached by a different client over its own TCP port.



Lab architecture · 2 clients, 1 TrueNAS server, 3 protocols

The Ubuntu VM reaches `pool-nfs` over NFS, while the Windows host reaches `pool-smb` over SMB and `pool-iscsi` over iSCSI. Because the Windows host sits on the same virtual switch, Wireshark on its VMnet8 adapter sees every iSCSI frame.

3.1 Addressing plan

Element	Value
Network	192.168.64.0/24, VMware NAT (VMnet8), DHCP
TrueNAS web UI	http://192.168.64.149 (login `truenas_admin`)
NFS export	`192.168.64.149:/mnt/pool-nfs/nfs-share`, allowed network 192.168.64.0/24
SMB share	`\\192.168.64.149\smb-share` (user `smbuser`)
iSCSI portal	0.0.0.0:3260 (all interfaces)
iSCSI target IQN	`iqn.2005-10.org.freenas.ctl:iscsi-target`

3.2 Disk allocation

Linux name	Size	Used by	Role
sda	20 GiB	boot-pool	TrueNAS operating system
sdb, sdc	2 × 10 GiB	pool-nfs	Mirror (RAID 1)
sdd, sde, sdf	3 × 10 GiB	pool-smb	RAIDZ1 (similar to RAID 5)
sdg, sdh	2 × 10 GiB	pool-iscsi	Mirror (RAID 1)

Using a different RAID layout for the SMB pool gives a direct comparison between mirroring and parity in the capacity analysis (section 5).

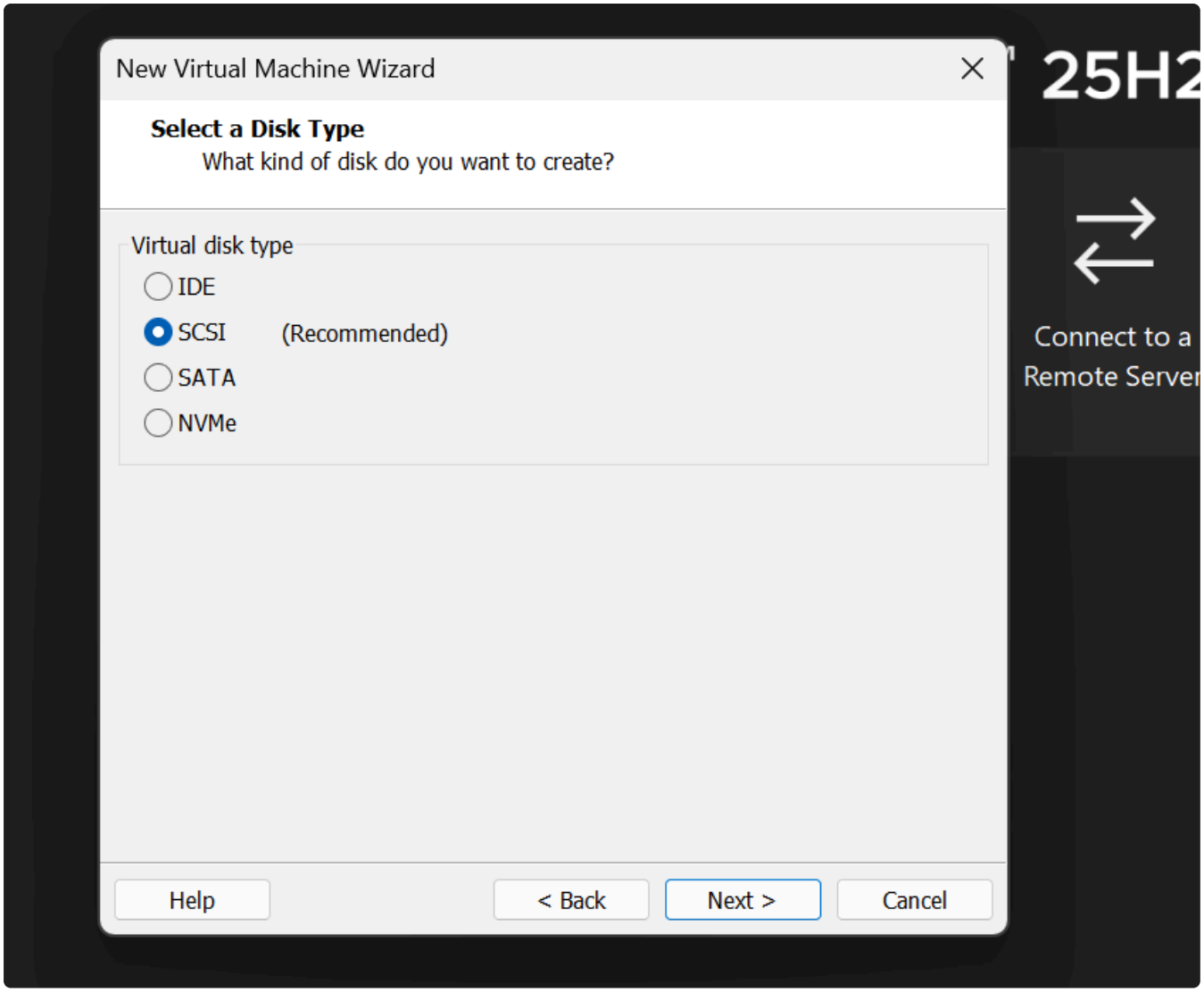
4. Part 1: Preparing the TrueNAS server

TrueNAS was installed on a dedicated VMware VM with one system disk and seven raw data disks. After installation, the web interface was reachable at http://192.168.64.149 and all seven data disks appeared unassigned, ready for ZFS.

4.1 Creating the virtual machine

The VM was created with the VMware *Custom (advanced)* wizard, using these settings:

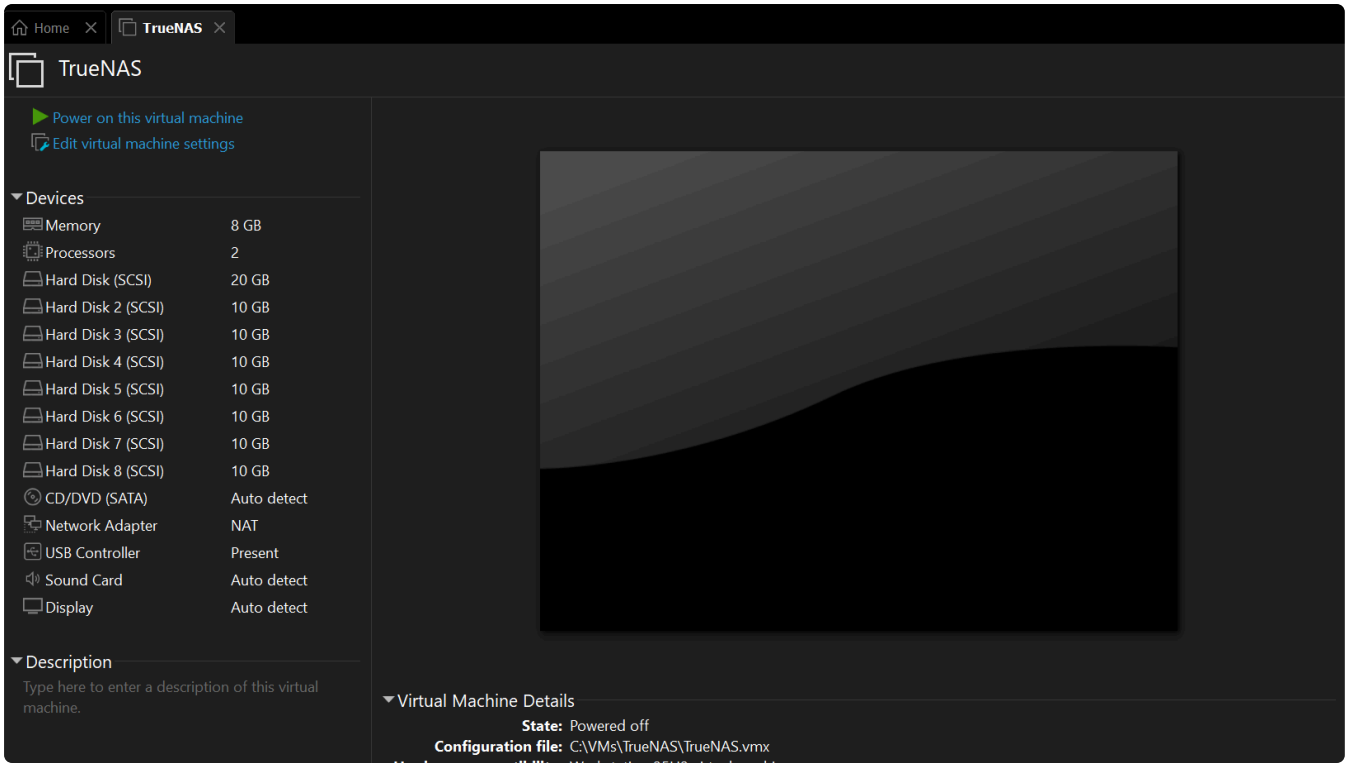
Setting	Value	Reason
Guest OS	Linux, Debian 12.x 64-bit	TrueNAS Community Edition is built on Debian, so VMware loads the right drivers
Firmware	UEFI	Modern boot standard
CPU	1 socket, 2 cores	Enough for ZFS checksums and LZ4 compression in a lab
Memory	8 GB	ZFS uses RAM as a read cache (the ARC); 8 GB is the TrueNAS recommendation
Network	NAT (VMnet8)	Puts TrueNAS on the same virtual network as the Windows host and the Ubuntu VM
Disk controller	SCSI (LSI Logic)	VMware's recommended type for Linux guests
Boot disk	20 GB, single file	Holds only the operating system



The VM folder was first created inside OneDrive, which would have tried to upload the virtual disks over the network. It was moved to `C:\VMs\TrueNAS` before any data was written (see section 12).

4.2 Adding the data disks

Seven extra disks of 10 GB each were added, all on the SCSI controller and thin-provisioned (*Allocate all disk space now* unchecked), so each file only grows as data is written. These seven disks play the role of the physical drives in a real NAS. They are raw: no partition, no file system, no pool.

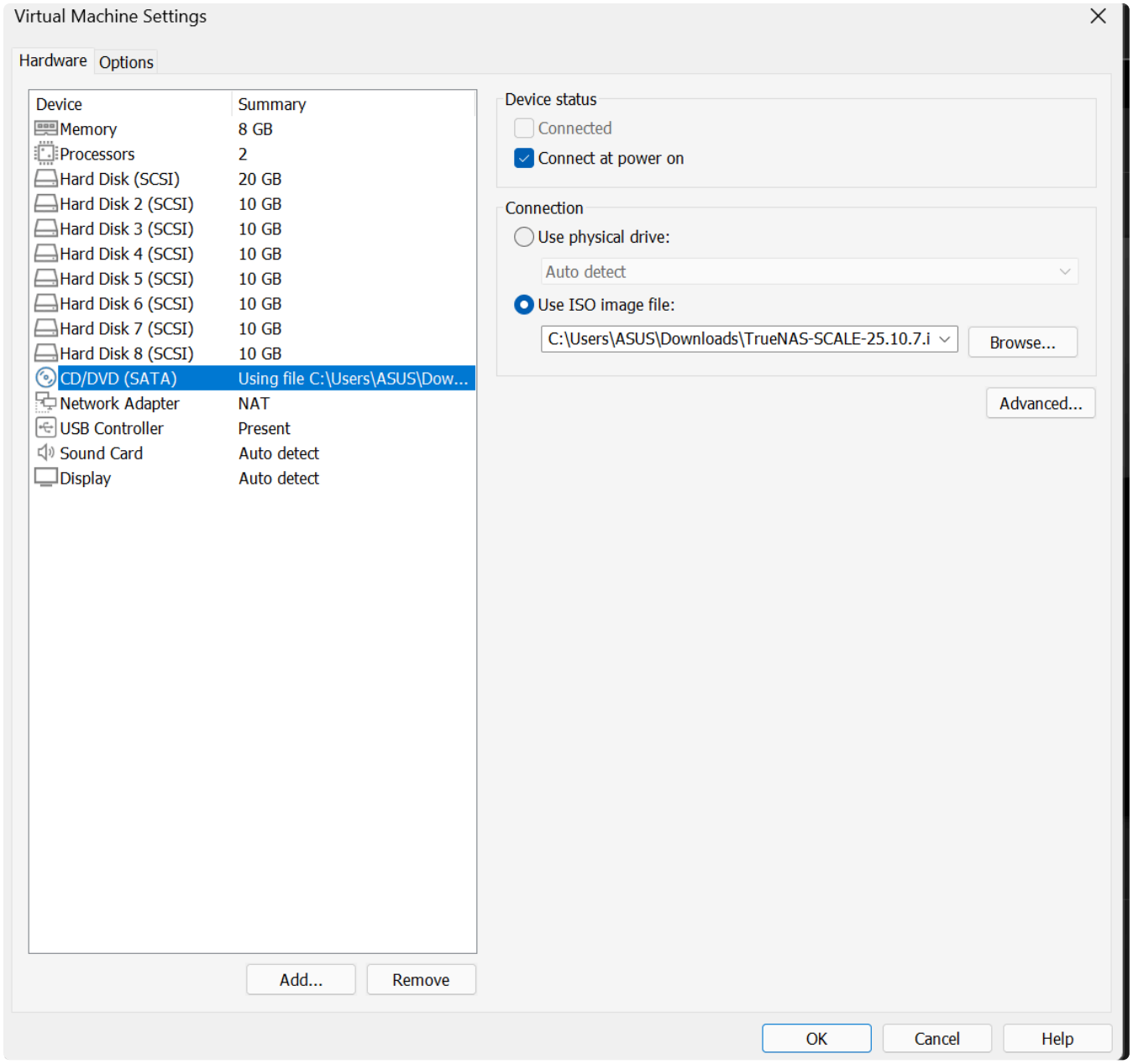


Two lines were then added to the VM configuration file `TrueNAS.vmx` with Notepad, while VMware was closed:

```
disk.EnableUUID = "TRUE"
firmware = "efi"
```

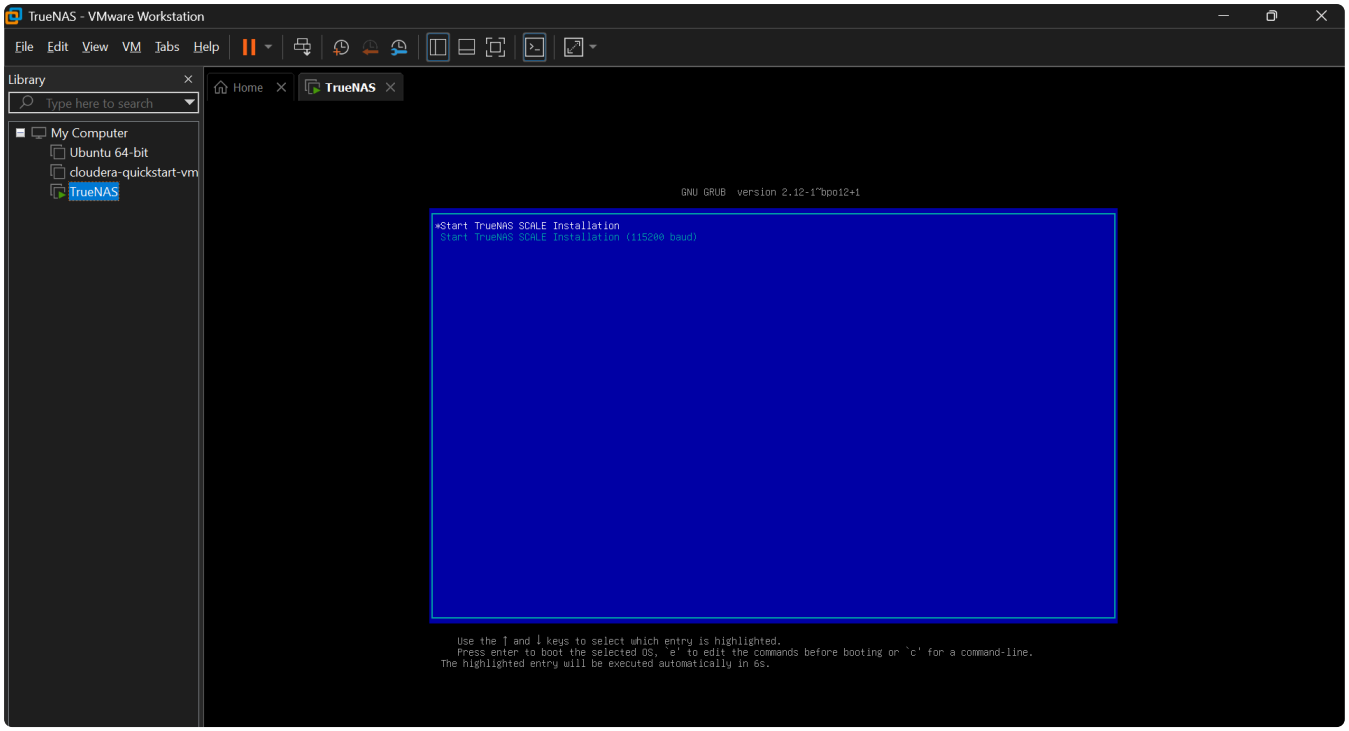
- `disk.EnableUUID = "TRUE"` asks VMware to expose a unique identifier for each virtual disk, so the guest can tell the disks apart.
- `firmware = "efi"` makes the VM boot in UEFI mode, since the original file had no firmware line and would have defaulted to legacy BIOS.

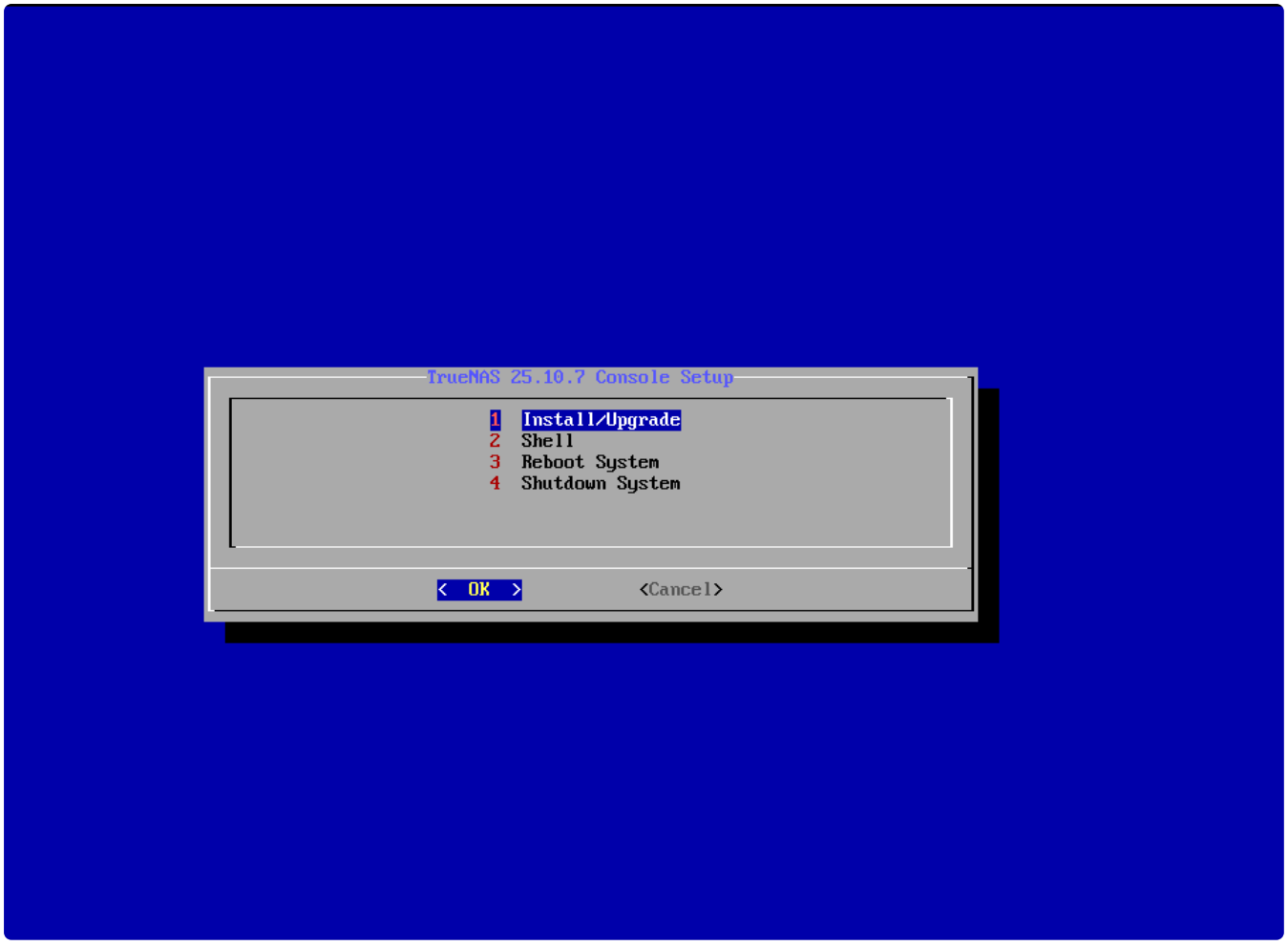
Finally, the TrueNAS ISO was attached to the virtual CD/DVD drive with *Connect at power on* checked.



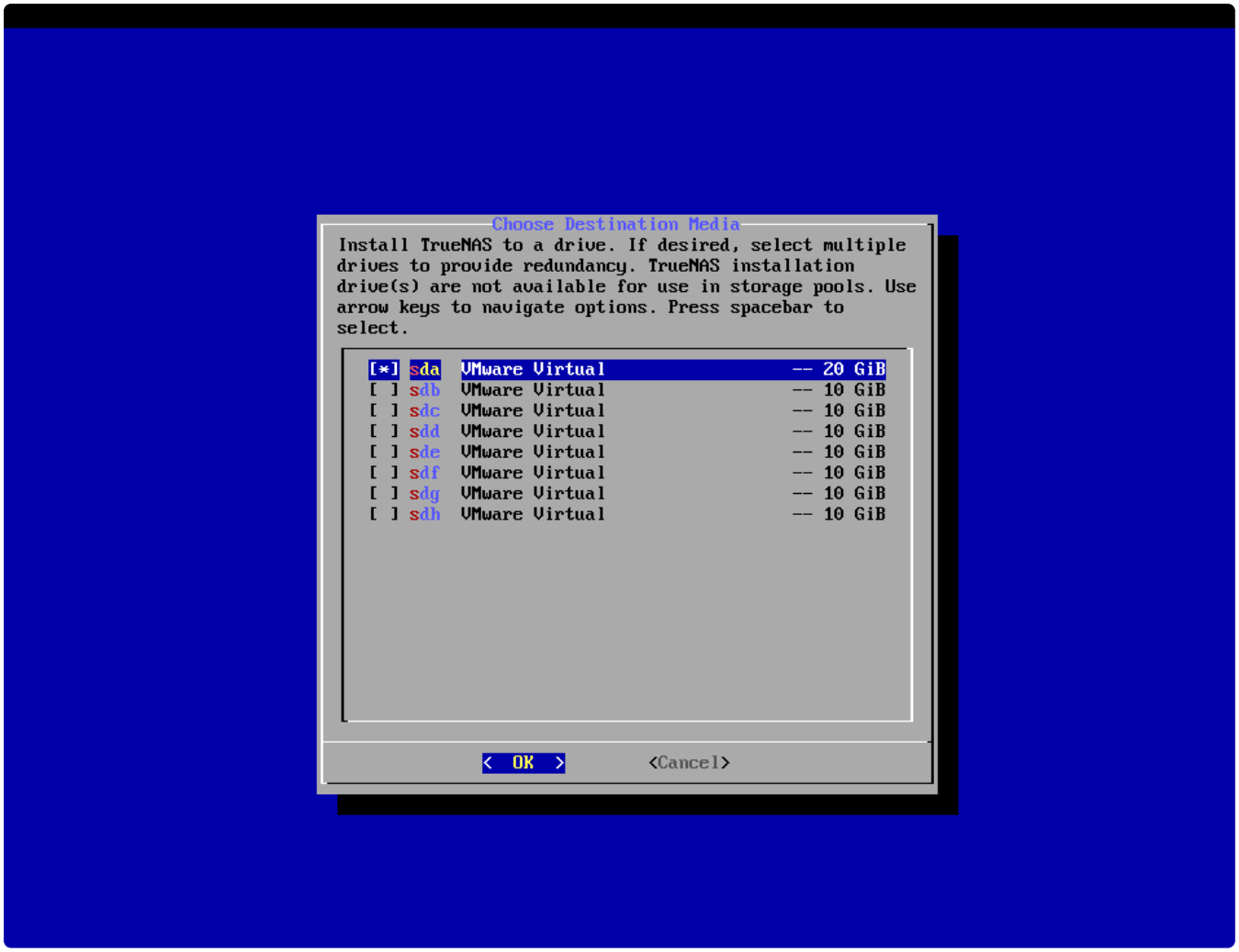
4.3 Installing TrueNAS

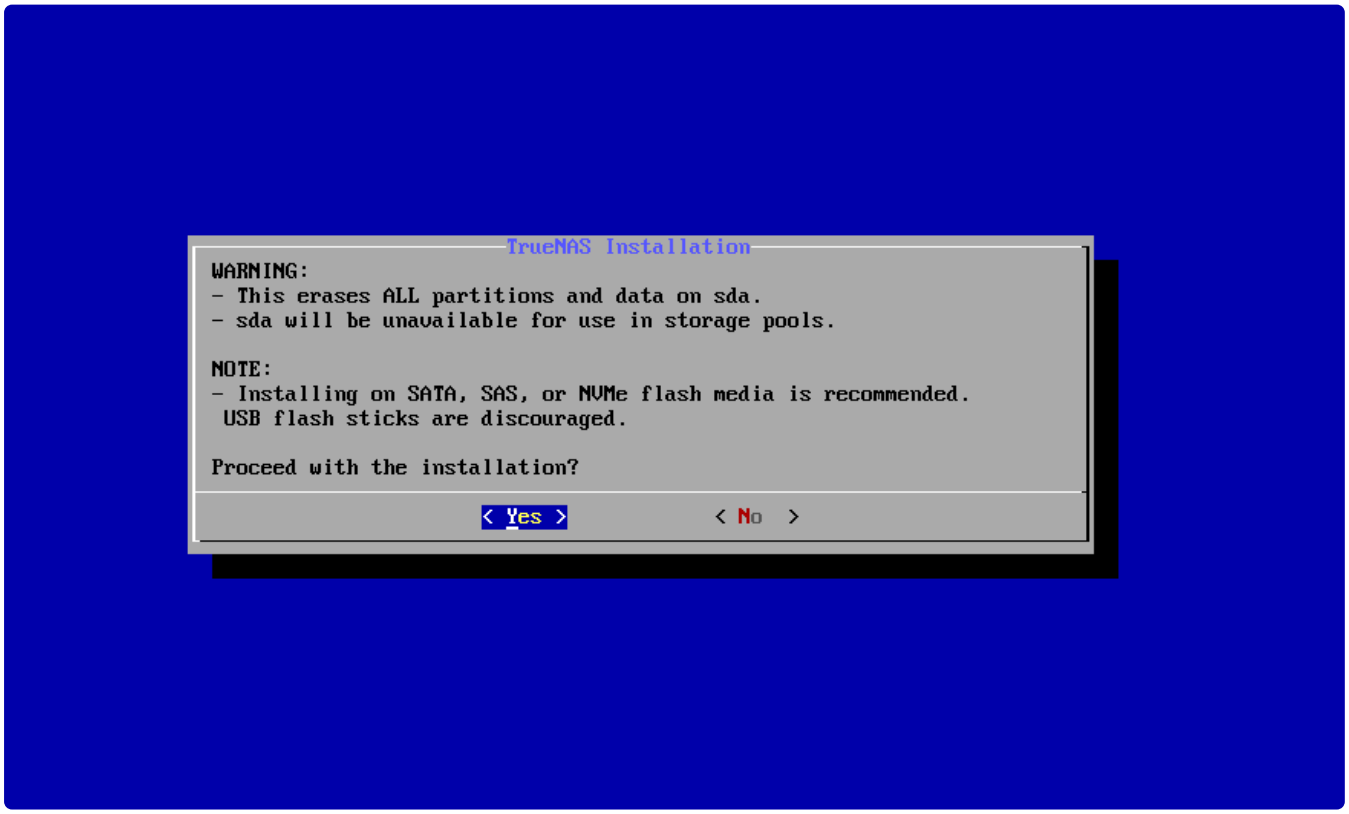
The VM booted from the ISO into the GRUB menu, and the default entry *Start TrueNAS SCALE Installation* was chosen. The second entry (115200 baud) is only for installing through a serial console.



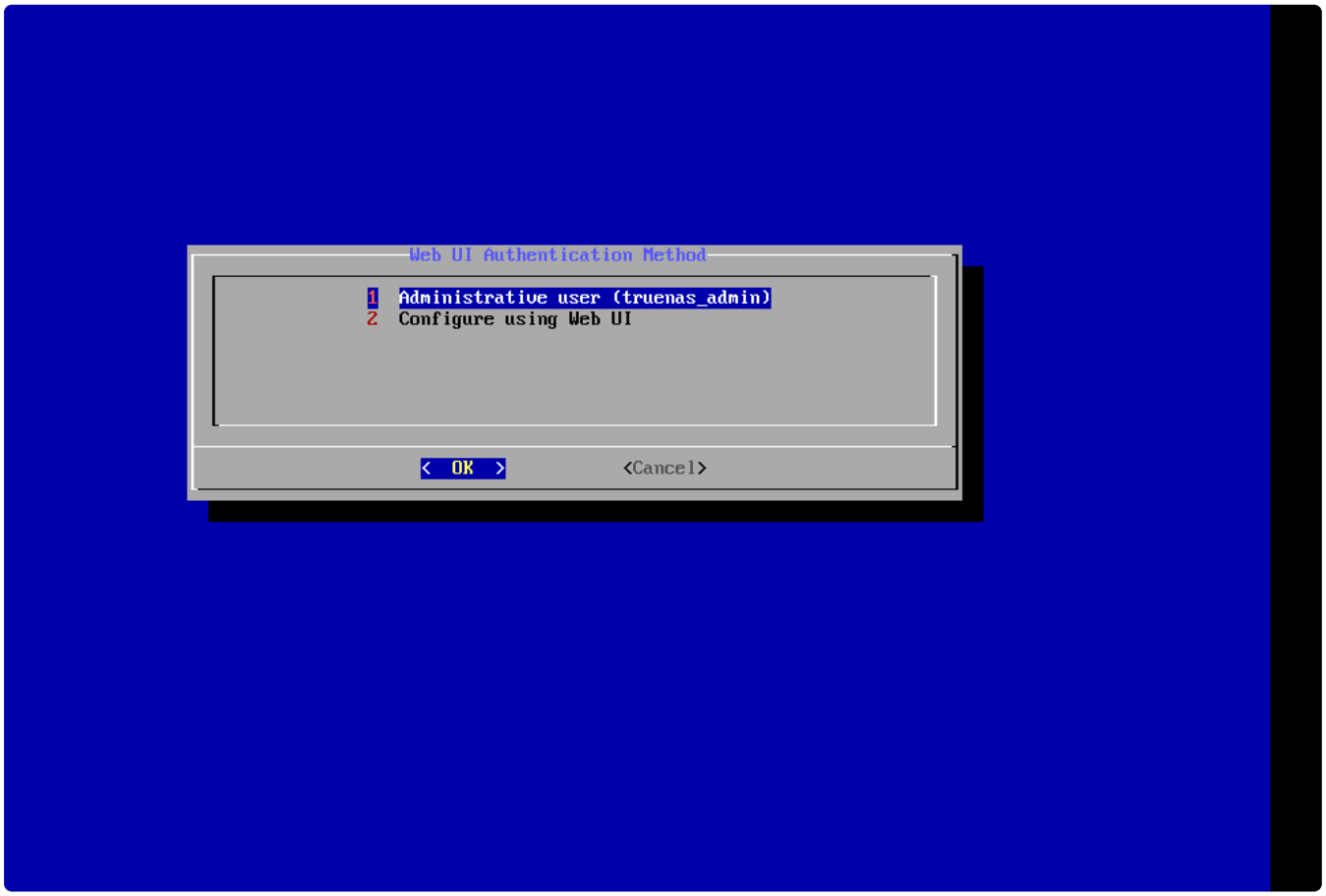


The installer then listed all eight disks with their Linux names. Only `sda` (20 GiB) was selected as the destination. The seven 10 GiB disks were left unselected, because a disk used for the system becomes unavailable for storage pools.

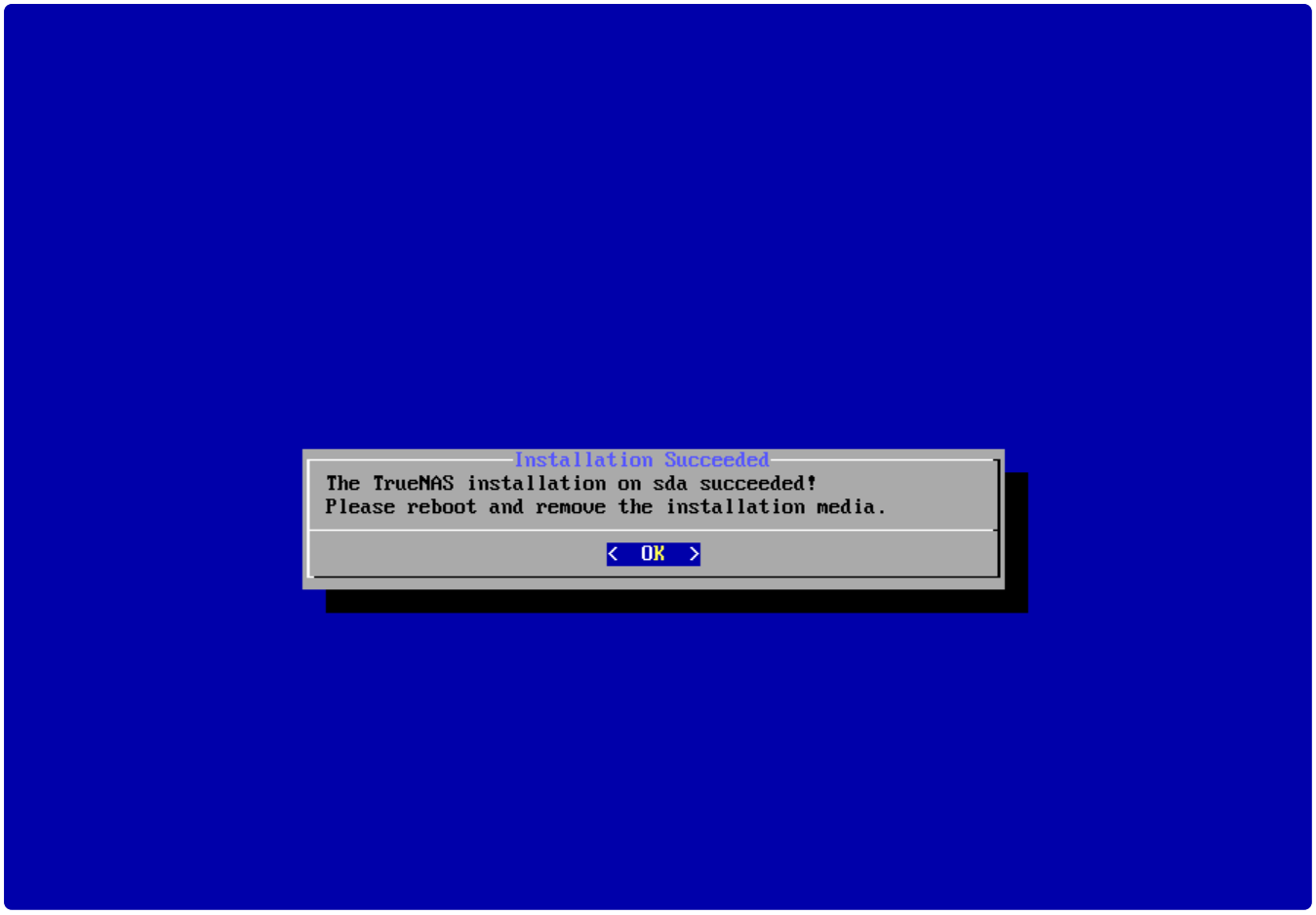

Choose Destination Media
Install TrueNAS to a drive. If desired, select multiple drives to provide redundancy. TrueNAS installation drive(s) are not available for use in storage pools. Use arrow keys to navigate options. Press spacebar to select.
[*] sda VMware Virtual -- 20 GiB
[ ] sdb VMware Virtual -- 10 GiB
[ ] sdc VMware Virtual -- 10 GiB
[ ] sdd VMware Virtual -- 10 GiB
[ ] sde VMware Virtual -- 10 GiB
[ ] sdf VMware Virtual -- 10 GiB
[ ] sdg VMware Virtual -- 10 GiB
[ ] sdh VMware Virtual -- 10 GiB
< OK >
<Cancel>


TrueNAS Installation
WARNING:
- This erases ALL partitions and data on sda.
- sda will be unavailable for use in storage pools.
NOTE:
- Installing on SATA, SAS, or NVMe flash media is recommended.
USB flash sticks are discouraged.
Proceed with the installation?
< Yes >
< No >

For web authentication, option *Administrative user (truenas_admin)* was chosen and a password was set. Since recent versions, TrueNAS disables password login for `root` and uses a named administrator account instead, which removes a well-known target for attackers.



The installer created a small ZFS pool called `boot-pool` on `sda` and extracted the system image into it, then reported success.

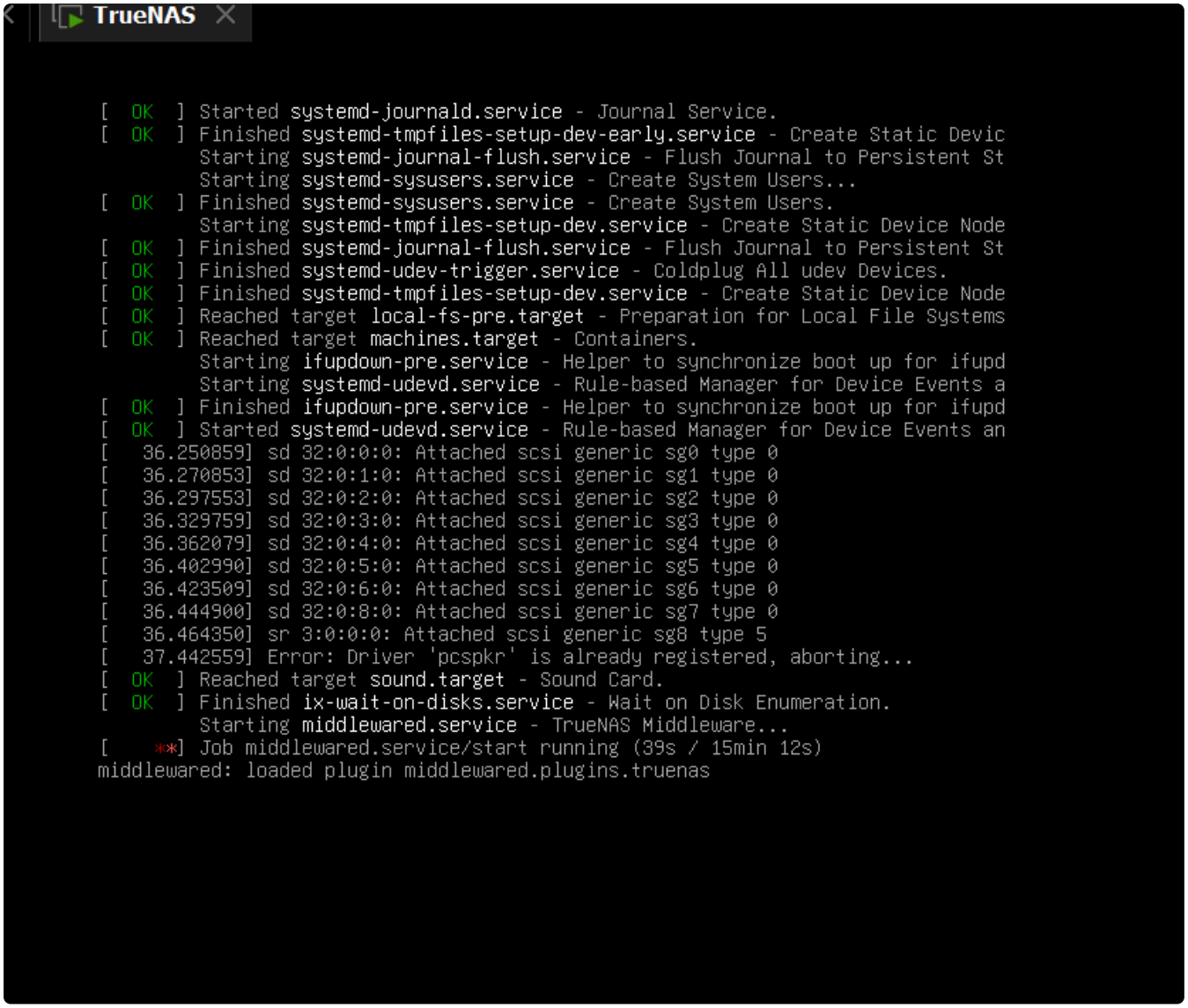


Before rebooting, the ISO was disconnected (VM > Removable Devices > CD/DVD > Disconnect, forcing past the guest's CD-ROM lock) so the VM would boot from `sda` instead of the installer. The new GRUB menu showed *TrueNAS Scale GNU/Linux 25.10.7* and a *UEFI Firmware Settings* entry, confirming the UEFI change worked.

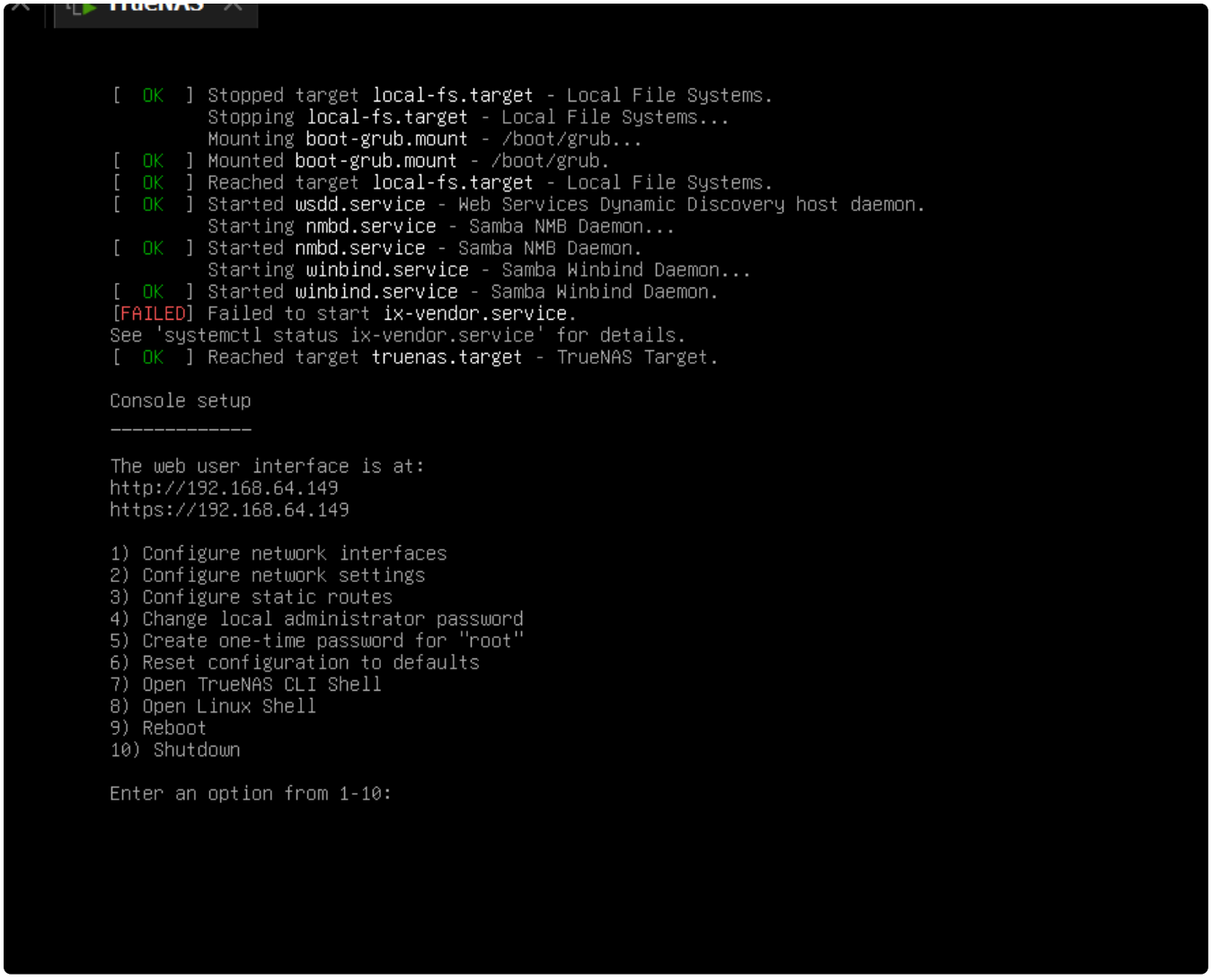


4.4 First boot

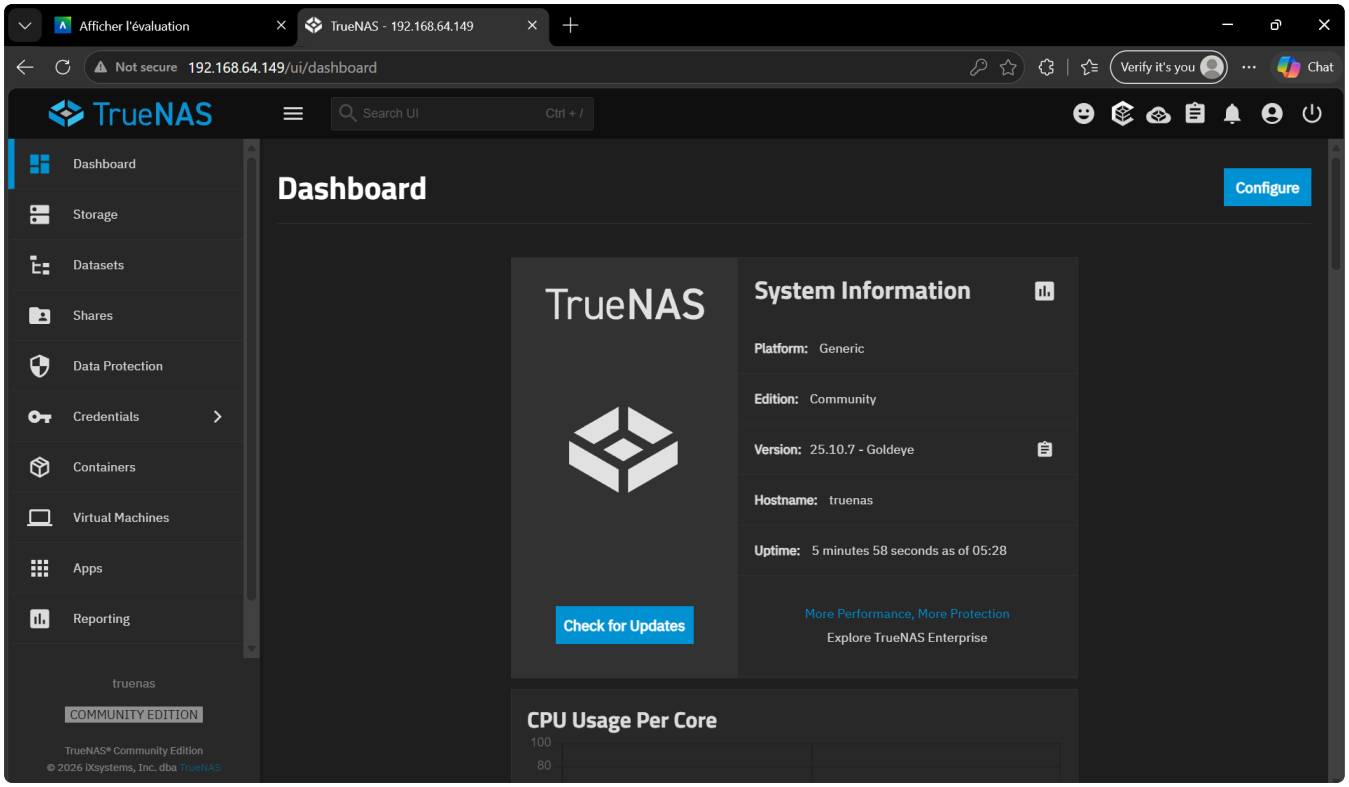
During the first boot, the kernel detected the eight SCSI disks as `sd 32:0:0:0` to `sd 32:0:7:0`. This SCSI address has the form *host:channel:target:LUN*, the same Target and LUN concepts used later by iSCSI. The `middlewared` service, which runs the web UI and the API, takes a few minutes to initialise the first time.



The console then displayed the web interface address, obtained from VMware's DHCP server. The `[FAILED] ix-vendor.service` line is harmless: that service configures iXsystems-branded hardware, which a VM is not. The `nmbd`, `winbind` and `wsdd` lines are the Samba services used later for SMB.

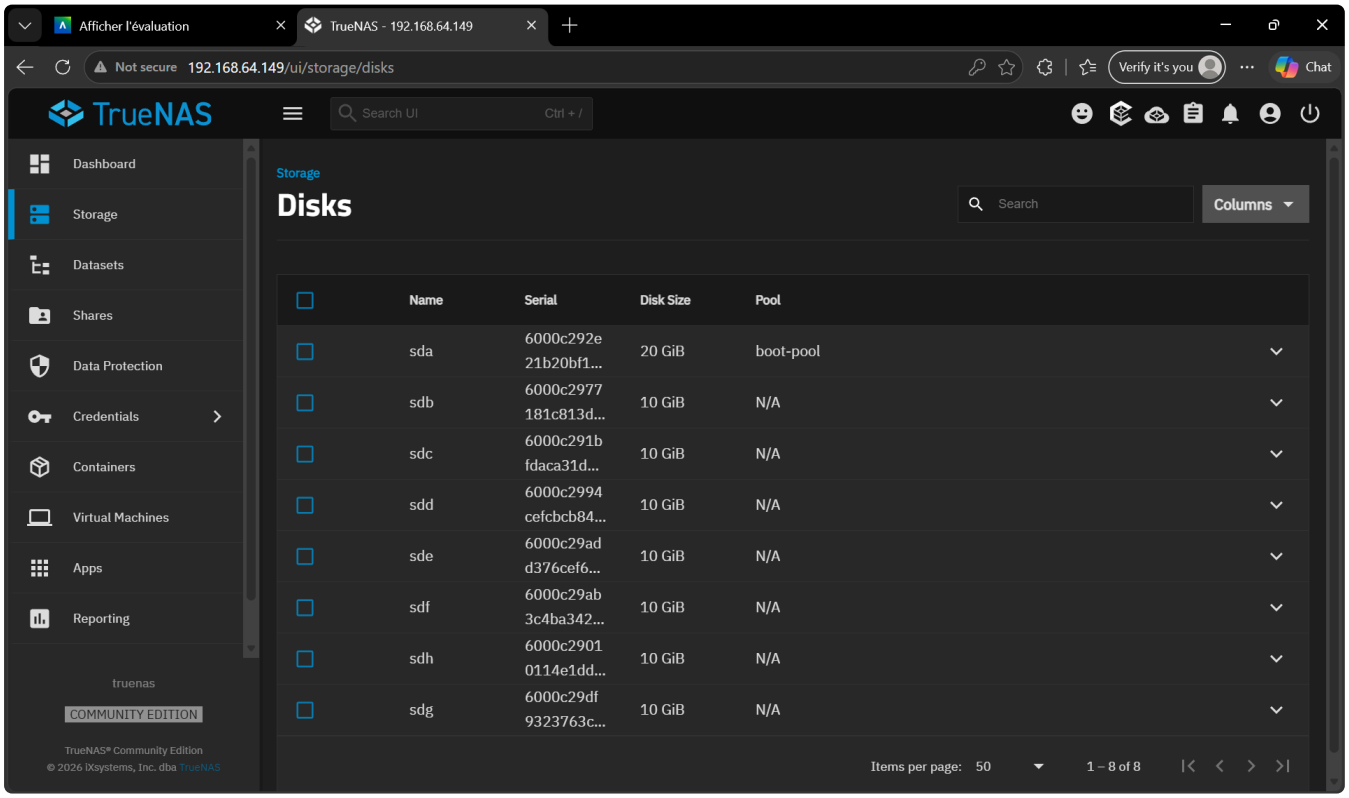


The login page and dashboard loaded from the Windows browser, proving that the host can reach the VM over VMnet8. The dashboard confirms the version (25.10.7, Goldeye), the Community edition and the hostname `truenas`. The connection used plain HTTP, which is acceptable on a private lab network; production would use HTTPS.



4.5 Identifying the raw disks

Storage > Disks lists eight disks. `sda` belongs to `boot-pool`; `sdb` to `sdh` show **Pool: N/A**, meaning they are raw and unassigned. Each disk shows an identifier in the Serial column (`6000c29…`), provided by the `disk.EnableUUID` setting.



A raw disk has no partition table, no file system and no pool membership, so TrueNAS sees it as free capacity. ZFS only partitions a disk when it is added to a pool (each member later appears as `sdb1`, `sdc1`, and so on).

5. Part 2: Storage configuration (VDEVs, RAID and pools)

The seven data disks were grouped into three ZFS pools: two 2-disk mirrors and one 3-disk RAIDZ1. All three pools came up ONLINE with no errors, giving 9.2 GiB, 19.03 GiB and 9.2 GiB of usable space.

5.1 ZFS vocabulary

ZFS replaces the classic stack of hardware RAID card, volume manager and file system with one layered model:

Term	What it is	In this lab
Disk	A raw physical (here virtual) drive	`sdb` to `sdh`
RAID	The technique of combining disks for redundancy or performance	Mirror, RAIDZ1
VDEV (virtual device)	A group of disks with one redundancy layout; the RAID level belongs to the VDEV	`mirror-0`, `raidz1-0`
Pool (zpool)	The storage space built from one or more VDEVs; data is striped across VDEVs	`pool-nfs`, `pool-smb`, `pool-iscsi`
Dataset	A file system inside a pool: files, folders, permissions, quotas	`nfs-share`, `smb-share`
Zvol	A raw block device carved out of a pool, with no file system	`zvol-iscsi`

The chain is therefore: **Disks → VDEV (RAID) → Pool → Dataset or Zvol**. A dataset serves files (NFS, SMB); a zvol serves a disk (iSCSI).

5.2 Building the first pool in the web UI

The Pool Creation Wizard (**Storage > Create Pool**) was used for `pool-nfs`. In *Manual Disk Selection*, a VDEV of type MIRROR was added and `sdb` and `sdc` were dragged into it. TrueNAS immediately estimated 10 GiB of usable capacity, because the second disk is a

full copy of the first. The optional VDEV classes (Log, Spare, Cache, Metadata, Dedup) were skipped.

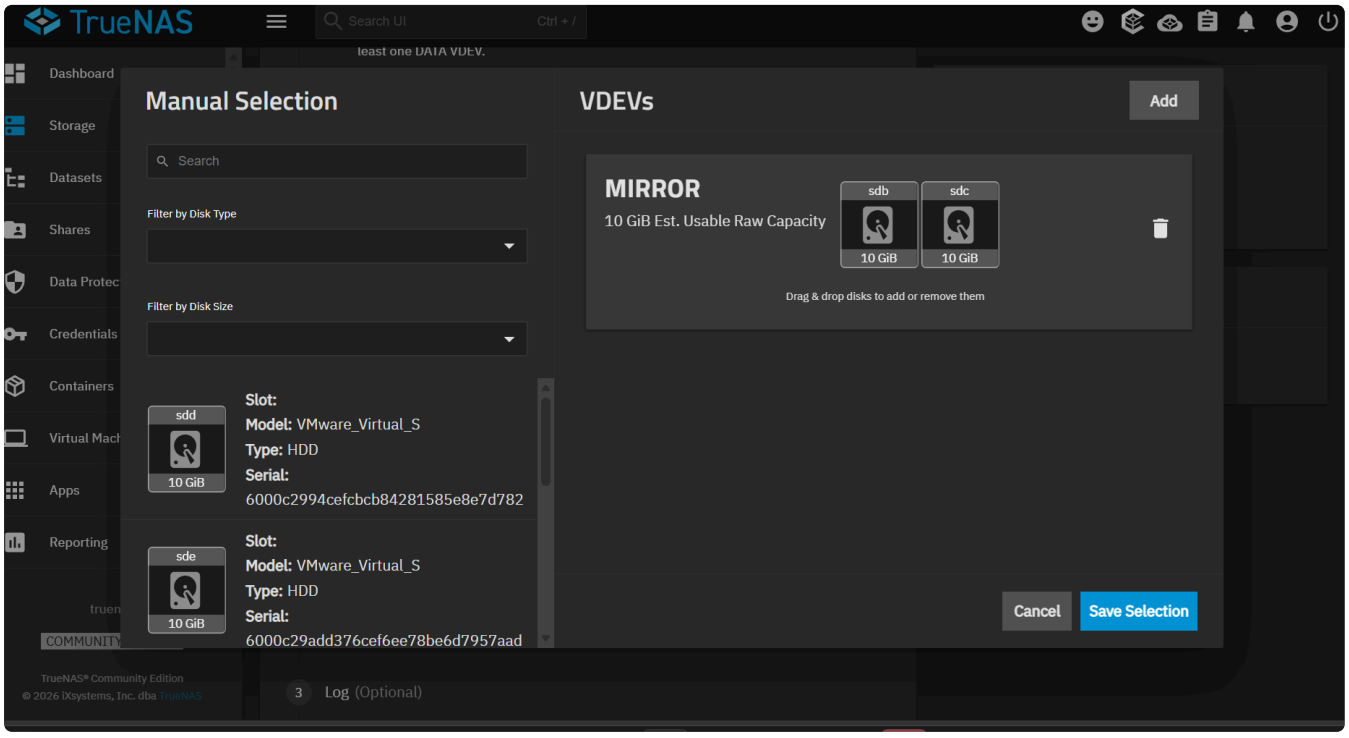


Image — Figure 17: Review step, pool-nfs with 1 data VDEV and 10 GiB usable

The optional VDEV classes are worth knowing:

- **Log (SLOG):** a fast device that speeds up synchronous writes.
- **Spare:** a standby disk that automatically replaces a failed one.
- **Cache (L2ARC):** an SSD read cache that extends the RAM cache.
- **Metadata / Dedup:** fast disks dedicated to metadata or deduplication tables.

Pool creation was refused with *Disks have duplicate serial numbers: None (sda, sdb, sdc)*. VMware's emulated controller returned an empty SCSI serial for every disk, so TrueNAS saw several disks with the same serial "None" and blocked the pool as a safety measure. Switching the controller to VMware Paravirtual did not change this.

Image — Figure 18: Pool creation refused, duplicate serial numbers None (sda, sdb, sdc)

5.3 Creating the pools through the TrueNAS API

The pools were therefore created from **System > Shell** with `midclt`, the TrueNAS middleware client. It calls the same API as the web UI, but accepts the option `allow_duplicate_serials`, which skips the serial-number check. This is safe here

because the seven disks are genuinely separate virtual disks; on real hardware the check should never be bypassed.

Two small issues had to be solved first (detailed in section 12): the browser shell inserted bracketed-paste markers (`^[[200~`) around pasted text, and the job flag is `-j` , not `-job` .

Image — Figure 19: Pasted command rejected by zsh because of bracketed-paste markers

Image — Figure 20: After unset zle_bracketed_paste, midclt rejects the -job flag

The three commands used were:

```
sudo midclt call -j pool.create '{"name": "pool-nfs", "topology": {"data":
[{"type": "MIRROR", "disks": ["sdb", "sdc"]}]}, "allow_duplicate_serials":
true}'
sudo midclt call -j pool.create '{"name": "pool-smb", "topology": {"data":
[{"type": "RAIDZ1", "disks": ["sdd", "sde", "sdf"]}]},
"allow_duplicate_serials": true}'
sudo midclt call -j pool.create '{"name": "pool-iscsi", "topology": {"data":
[{"type": "MIRROR", "disks": ["sdg", "sdh"]}]}, "allow_duplicate_serials":
true}'
```

`pool.create` builds the pool, `topology.data` describes its data VDEVs, and `-j` waits for the background job to finish and prints the result as JSON.

Image — Figure 21: pool-nfs created, MIRROR VDEV mirror-0 with sdb1 and sdc1, ONLINE an…

The JSON output contains the key properties of each pool:

Field	Value for pool-nfs	Meaning
`path`	`/mnt/pool-nfs`	Mount point of the pool
`type` , `name`	MIRROR, `mirror-0`	The VDEV and its RAID layout
`disk` / `device`	`sdb` / `sdb1` , `sdc` / `sdc1`	TrueNAS partitioned each disk and gave the partition to ZFS
`status` , `healthy`	ONLINE, true	Both members working

Field	Value for pool-nfs	Meaning
`size`	10,200,547,328 bytes (about 9.5 GiB)	Pool size after ZFS overhead
`logical_ashift`	12	Block size 2^{12} = 4 KiB, the modern sector size

Image — Figure 22: pool-smb created, RAIDZ1 VDEV raidz1-0 with sdd1, sde1 and sdf1

For RAIDZ1, the reported `size` (31,675,383,808 bytes, about 29.5 GiB) is the raw size of all three disks, parity included. For a mirror, the reported size already excludes the copy. This explains why the RAIDZ1 number looks larger than its usable capacity.

Image — Figure 23: pool-iscsi created, MIRROR VDEV with sdg1 and sdh1

5.4 Capacity and fault tolerance

Image — Figure 24: Storage dashboard, the three pools ONLINE with no errors

Pool	Disks	Layout	Theoretical usable	Usable shown by TrueNAS	Efficiency	Fault tolerance
pool-nfs	sdb, sdc	1 × MIRROR, 2 wide	10 GiB	9.2 GiB	50 %	1 disk
pool-smb	sdd, sde, sdf	1 × RAIDZ1, 3 wide	20 GiB	19.03 GiB	67 %	1 disk
pool-iscsi	sdg, sdh	1 × MIRROR, 2 wide	10 GiB	9.2 GiB	50 %	1 disk

The theoretical capacities follow from the layouts:

- **Mirror (similar to RAID 1):** every block is written identically on both disks. Usable = 1 disk = 10 GiB. It survives the loss of one disk, because the other holds a full copy. Reads can be served by either disk.
- **RAIDZ1 (similar to RAID 5):** data is striped over the three disks with one parity block per stripe, rotated across disks. Usable = (n − 1) × size = 2 × 10 = 20 GiB. It survives the loss of one disk; the missing data is rebuilt from parity. It uses space better than a mirror, but rebuilding after a failure is slower and reads every remaining disk.

The gap between theoretical and observed capacity (about 4–8 %) comes from three sources: the GiB/GB difference in VMware's disk sizes, ZFS metadata and partition layout, and the *slop space*, a small reserve ZFS keeps so the pool can still operate when it is nearly full.

6. Part 3: NFS share for a Linux client

The dataset `pool-nfs/nfs-share` was exported over NFS and mounted on the Ubuntu VM with NFSv4.2. Files created, modified, copied and deleted on Ubuntu appeared and disappeared identically on the TrueNAS server.

6.1 Creating the dataset

In **Datasets**, `pool-nfs` was selected and **Add Dataset** was used with name `nfs-share` and preset **Generic**. An NFS share exports a directory, so it needs a file system: the dataset is that file system, mounted on the server at `/mnt/pool-nfs/nfs-share`.

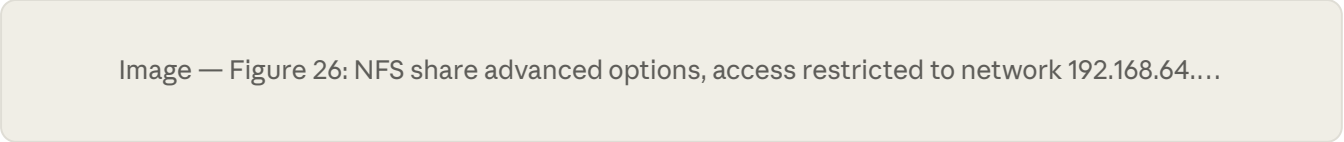
Image — Figure 25: Dataset pool-nfs/nfs-share, LZ4 compression, case sensitivity ON

The dataset details show **Compression: LZ4** (ZFS compresses data on the fly at very low CPU cost) and **Case Sensitivity: ON**, the Linux convention. The dataset shares the pool's 9.2 GiB because no quota was set.

6.2 Creating the NFS share

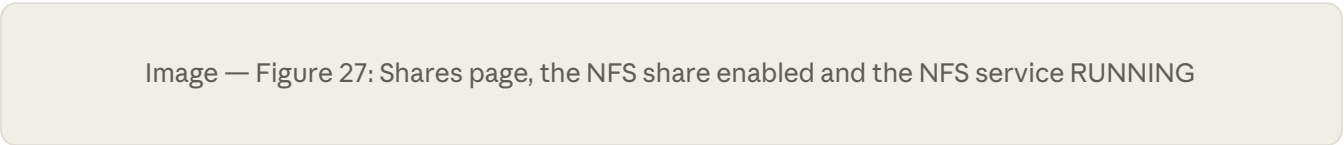
In **Shares > UNIX (NFS) Shares > Add**, the share was configured as follows:

Option	Value	Purpose
Path	`/mnt/pool-nfs/nfs-share`	Directory to export
Description	Partage NFS mini-projet	Label
Networks	192.168.64.0/24	Only hosts on the lab network may mount the share
Mapall User / Group	root / root	Every client user is mapped to root on the server, so writes are allowed
Read Only	unchecked	Clients can write



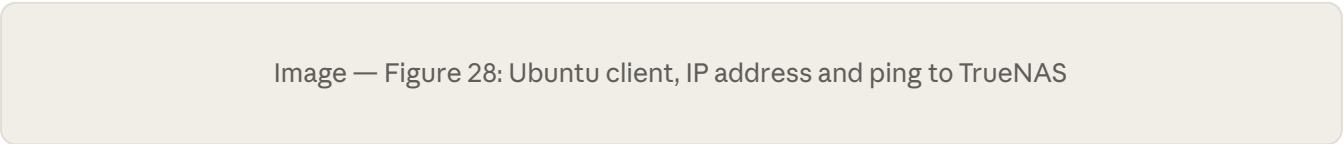
NFS with the default `sec=sys` security does not use passwords: it trusts the numeric UID and GID sent by the client. The Ubuntu user (UID 1000) does not exist on TrueNAS, so without mapping it could not write to the dataset. The form first had **Maproot** filled in, which only maps the client's root user; it was changed to **Mapall**, which maps every client user. Mapall is convenient in a lab; in production, matching user accounts or proper ACLs would be used instead.

The NFS service was then started with *start automatically* enabled, so it comes back after a reboot.



6.3 Preparing the Ubuntu client

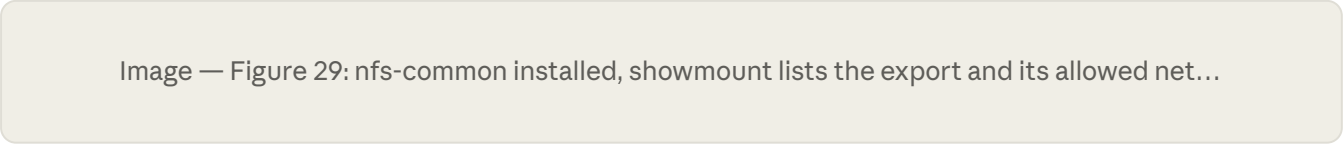
On Ubuntu, `ip a` showed the address **192.168.64.147/24** on interface `ens33`, on the same network as TrueNAS. `ping` confirmed connectivity: 4 packets sent, 4 received, 0 % loss, average 0.88 ms.



The NFS client tools were installed and the server's export list was queried:

```
sudo apt update && sudo apt install -y nfs-common
showmount -e 192.168.64.149
```

The package `nfs-common` provides `mount.nfs` and `showmount`, and pulled in `rpcbind`. NFS is built on RPC (Remote Procedure Call), and `rpcbind` tells clients which port each RPC service uses. The export list returned exactly one entry: `/mnt/pool-nfs/nfs-share 192.168.64.0/24`.



6.4 Mounting the share

```
sudo mkdir -p /mnt/nfs
sudo mount -t nfs 192.168.64.149:/mnt/pool-nfs/nfs-share /mnt/nfs
df -h /mnt/nfs
mount | grep nfs
```

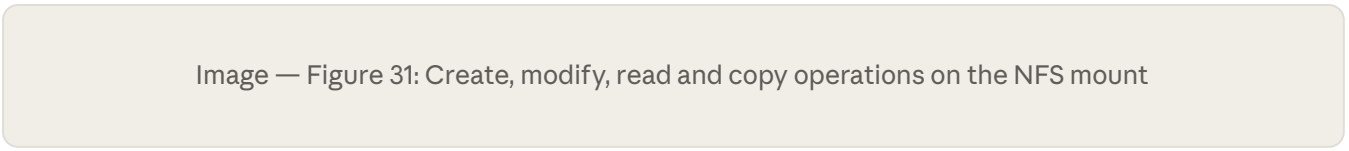
Image — Figure 30: Share mounted on /mnt/nfs, 9.3G, NFSv4.2 over TCP

Mount option	Meaning
`type nfs4`, `vers=4.2`	The newest NFS version, negotiated automatically
`rw`	Read-write access
`proto=tcp`	NFSv4 runs over TCP, on port 2049
`rsize=1048576`, `wsize=1048576`	Up to 1 MiB per read or write request
`hard`	If the server stops answering, the client waits and retries instead of returning errors
`sec=sys`	AUTH_SYS: identification by UID/GID, which is why Mapall was needed
`clientaddr=192.168.64.147`, `addr=192.168.64.149`	Client and server addresses

After mounting, the remote directory is attached to the local folder `/mnt/nfs`. Any program on Ubuntu now reads and writes there as if it were local, while the bytes are stored on the TrueNAS ZFS mirror.

6.5 File operations from the client

```
cd /mnt/nfs
echo "Bonjour depuis le client NFS Ubuntu" > test.txt   # create
mkdir dossier1                                          # create a folder
echo "Ligne ajoutee" >> test.txt                          # modify
cat test.txt                                            # read
cp test.txt dossier1/copie.txt                          # copy
ls -l /mnt/nfs /mnt/nfs/dossier1
```

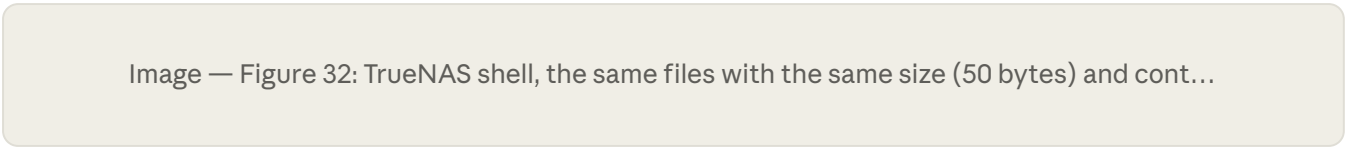


All files are owned by `root root` even though they were created by the user `islem9antra`. This is the Mapall rule in action. Running `ls /mnt/pool-nfs/...` on Ubuntu fails, because the client only sees the share through its mount point, never the server's real path.

6.6 Verification on the server

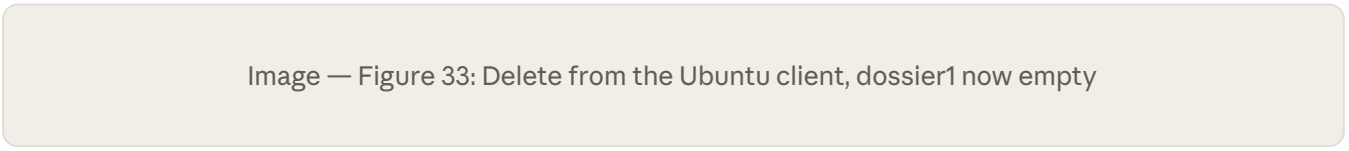
The same listing was run in the TrueNAS shell, on the real path:

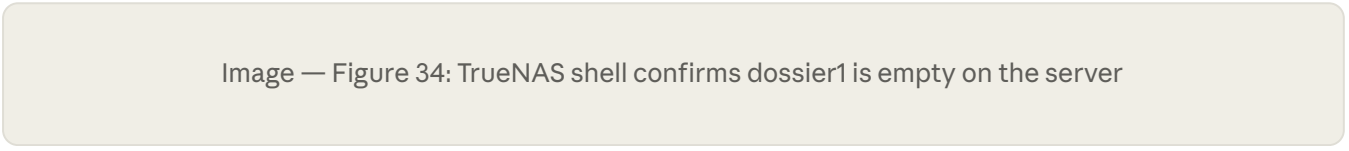
```
ls -l /mnt/pool-nfs/nfs-share /mnt/pool-nfs/nfs-share/dossier1
cat /mnt/pool-nfs/nfs-share/test.txt
```



The server shows `dossier1`, `test.txt` and `copie.txt` with the same 50-byte size and both lines of text. The times differ by eight hours only because TrueNAS uses its default US/Pacific timezone.

Finally, `copie.txt` was deleted from the client with `rm /mnt/nfs/dossier1/copie.txt`. Both the client and the server then reported `total 0` for `dossier1`.





Conclusion for NFS: the client sends file operations (open, read, write, remove) over TCP port 2049, and TrueNAS executes them on its own ZFS file system. This is file-level access: the client never manages the disk or its file system.

7. Part 4: SMB share for a Windows client

The dataset `pool-smb/smb-share` was shared over SMB and opened from Windows File Explorer as the dedicated user `smbuser`. A text file, a folder and a 189 KB image written from Windows were found on the server, owned by `smbuser`.

7.1 Creating a dedicated SMB user

Unlike NFS with `sec=sys`, SMB authenticates every connection with a username and password. A local user was created in **Credentials > Users > Add**:

Field	Value	Reason
Username	`smbuser`	Account used by the Windows client
Full Name	Utilisateur SMB	Label
SMB Access	checked	Stores the password in the format Samba needs
TrueNAS / Shell / SSH Access	unchecked	The account can access shares only, not administer the NAS
Group	new `smbuser` group	Personal primary group
Home Directory	`/var/empty`	No home folder needed

Image — Figure 35: Add User, smbuser with SMB access only

Giving the account only what it needs is the principle of least privilege. The Users list compares the three accounts: `root` (built-in, no password, login disabled), `truenas_admin` (full administrator) and `smbuser` (SMB only).

Image — Figure 36: Users list, root, truenas_admin and smbuser

7.2 Creating the dataset and the share

In **Datasets**, `pool-smb` was selected and **Add Dataset** was used with name `smb-share`, preset **SMB**, and *Create SMB Share* checked with share name `smb-share`. The SMB preset configures the dataset the way Windows expects. The first attempt did not create the dataset; the second, saved with more patience, did.

Image — Figure 37: Add Dataset, preset SMB with Create SMB Share checked

Image — Figure 38: Dataset pool-smb/smb-share, case sensitivity OFF

The two datasets differ in one important property:

Property	nfs-share	smb-share
Preset	Generic	SMB
Case sensitivity	ON (Linux convention)	OFF (Windows convention)
Pool	Mirror, 9.2 GiB	RAIDZ1, 19.03 GiB

With case sensitivity OFF, `Rapport.txt` and `rapport.txt` are the same file, exactly as on Windows. The SMB service was started with automatic start enabled.

7.3 Accessing the share from Windows

In File Explorer, the UNC path `\\192.168.64.149\smb-share` was typed in the address bar. Windows connected to TCP port 445, asked for credentials, and `smbuser` with its password was accepted. The empty share opened under **Network > 192.168.64.149 > smb-share**.

Image — Figure 39: Windows File Explorer connected to \192.168.64.149\smb-share

7.4 Read and write operations

From Windows:

1. **Write:** a text file was created with the content `Bonjour depuis Windows via SMB`, and a folder `dossier-smb` was created.
2. **Copy:** a 189 KB PNG screenshot was copied into `dossier-smb`.
3. **Read:** the text file and the image were opened again from the share.

Image — Figure 40: The 189 KB PNG copied into smb-share\dossier-smb

7.5 Verification on the server

```
ls -l /mnt/pool-smb/smb-share /mnt/pool-smb/smb-share/dossier-smb
cat /mnt/pool-smb/smb-share/test-smb.txt.txt
```

Image — Figure 41: TrueNAS shell, files owned by smbuser and the text written from Wind…

The results confirm the operations:

- `dossier-smb` and the PNG (193,406 bytes, matching the 189 KB shown by Windows) are on the server.
- Every file is owned by **`smbuser`**. SMB writes files as the authenticated user, unlike the NFS share where Mapall turned every user into root.
- `cat` returns `Bonjour depuis Windows via SMB`. The `%` after it is how zsh marks a file with no final newline, since Notepad does not add one.
- The file is named `test-smb.txt.txt`: Windows hides known extensions, so typing `test-smb.txt` added a second `.txt`.

Conclusion for SMB: like NFS, SMB is file-level. Windows sends file requests (create, read, write) over TCP 445 after authenticating, and TrueNAS, through Samba, executes them on its ZFS dataset.

8. Part 5: iSCSI block storage for Windows

A 5 GiB zvol on `pool-iscsi` was published as an iSCSI target and connected to the Windows iSCSI Initiator. Windows saw it as a new local disk, initialised it in GPT, formatted it NTFS as drive `I:` and stored files on it. Meanwhile, TrueNAS could not see those files at all.

8.1 Creating the zvol

In **Datasets**, `pool-iscsi` was selected and **Add Zvol** was used with name `zvol-iscsi` and size **5 GiB**. A zvol is a ZFS volume that behaves like a disk: a range of numbered blocks with no file system. The size leaves about 45 % of the pool free; ZFS performance drops when a pool is nearly full, and keeping at least 20 % free is the usual advice.

Image — Figure 42: zvol-iscsi created, 5.08 GiB reserved in pool-iscsi, 56 KiB written

The pool now shows 5.08 GiB used and 4.12 GiB available, although the zvol only contains 56 KiB of data. TrueNAS created a **thick-provisioned** zvol: the full size is reserved immediately, so the disk Windows sees can never run out of real space. A sparse (thin) zvol would consume space only as it is written, which is more flexible but allows overcommitment.

8.2 Configuring the target with the iSCSI Wizard

Shares > Block (iSCSI) Shares Targets > Wizard creates every iSCSI object in three steps:

Wizard step	Setting	Value
1. Target	Target	Create New
2. Extent	Name	`iscsi-target`
2. Extent	Extent Type	Device (a zvol), rather than File
2. Extent	Device	`pool-iscsi/zvol-iscsi (5 GiB)`
2. Extent	Sharing Platform	Modern OS (4K logical blocks, TPC enabled)
3. Protocol Options	Portal	New portal on 0.0.0.0, TCP port 3260
3. Protocol Options	Initiators	Empty, so all initiators are allowed

Image — Figure 43: iSCSI Wizard, extent iscsi-target on device pool-iscsi/zvol-iscsi

The Sharing Platform sets the logical block size seen by the client. The default *VMware* option uses 512-byte blocks for older ESXi hosts; *Modern OS* uses 4 KiB, matching the pool's 4 KiB blocks (`ashift 12`) and modern Windows. The Wireshark capture later confirms this: each SCSI block is 4,096 bytes. *TPC* (Third Party Copy) lets the client ask the server to copy blocks internally, without moving the data over the network.

After saving, all three sharing services were running.

Image — Figure 44: Shares page, SMB, NFS and iSCSI services all RUNNING

8.3 Target, Extent and their association

Image — Figure 45: Targets tab, iscsi-target with portal group 1, initiator group 1 (al…

Image — Figure 46: Extents tab, iscsi-target backed by zvol/pool-iscsi/zvol-iscsi, prod…

Image — Figure 47: Initiators tab, group 1 allows all initiators

The objects work together as follows:

Object	Role	In this lab
Extent	The storage that is shared	`zvol/pool-iscsi/zvol-iscsi`, with its own serial and NAA identifier
Target	The server endpoint a client logs in to, named by an IQN	`iqn.2005-10.org.freenas.ctl:iscsi-target`
Portal	IP address and TCP port the target listens on	0.0.0.0:3260
Initiator group	Which client IQNs may connect	All initiators
Associated target	The link between target and extent, which fixes the LUN number	Extent mapped to the target as LUN 0
Authentication	CHAP secret, optional	None

The IQN (iSCSI Qualified Name) has the form `iqn.<year-month>.<reversed domain>:<name>`. Here, `iqn.2005-10.org.freenas.ctl` is the TrueNAS base name and `iscsi-target` is the target name. Interestingly, the extent has a unique serial number, the very thing VMware's virtual disks were missing in section 5.

8.4 Connecting the Windows initiator

The Wireshark capture was started first (section 9), because the login exchange only happens at connection time. Then, in the Windows **iSCSI Initiator**, `192.168.64.149` was entered under *Quick Connect*. Windows discovered the target and logged in.

Image — Figure 48: Windows iSCSI Initiator, target iqn.2005-10.org.freenas.ctl:iscsi-ta…

The message *No Targets available for Login using Quick Connect* is not an error: the only target was already logged in. To record the full login in Wireshark, the target was then disconnected and connected again from the *Targets* tab.

8.5 Initialising and formatting the disk

In **Disk Management** (`diskmgmt.msc`), a new **Disk 1, 5.00 GB, Not Initialized** appeared beside the laptop's own Disk 0. It is the iSCSI LUN 0, seen by Windows exactly like a local disk. It was initialised with a **GPT** partition table.

Image — Figure 49: Disk Management, the 5 GB iSCSI LUN appears as Disk 1 and is initial…

A *New Simple Volume* was then created with the maximum size, drive letter `I:` , file system NTFS, default 4 KB allocation unit, label `iSCSI` and quick format. Windows compression was left off because ZFS already compresses with LZ4 on the server.

Image — Figure 50: Disk 1 formatted, iSCSI (I:), 4.98 GB NTFS, Healthy

The volume shows 4.98 GB rather than 5.00 GB because GPT initialisation reserves a small Microsoft Reserved partition. Windows also applied **BitLocker** automatically, because device encryption is enabled for fixed disks on this laptop. This is a useful property of block storage: the client can encrypt the disk, and the server only ever stores ciphertext.

In File Explorer, the new drive appears under **This PC** next to `C:` , not under **Network** like the SMB share.

Image — Figure 51: The iSCSI disk listed under This PC as a local drive

8.6 Read and write operations

A text file `test-iscsi.txt` and a 68 KB image were written to `I:` , then opened again to generate reads.

Image — Figure 52: Files written to the iSCSI drive I:

8.7 The block-level proof

On TrueNAS, the files written by Windows were searched for:

```
ls -la /mnt/pool-iscsi
ls -l /dev/zvol/pool-iscsi/
```

Image — Figure 53: TrueNAS cannot list the Windows files; the zvol is only a block devi…

`/mnt/pool-iscsi` is empty, and the zvol exists only as the block device `zvol-iscsi -> ../../zd0`. With NFS and SMB, the same `ls` on the server showed every file. With iSCSI, TrueNAS stores numbered blocks; the NTFS file system, and the files inside it, exist only from the Windows point of view.

9. Part 6: Wireshark capture and analysis

The capture on VMnet8 recorded 65,450 packets of iSCSI traffic. It contains the full life of a session: TCP connection, two-phase login, LUN discovery, then thousands of SCSI Read(10) and Write(10) commands carried in TCP segments.

9.1 Capture setup

Wireshark 4.6.9 was installed with Npcap 1.88, the Windows capture driver that copies each packet passing through a network interface. USBPcap and the optional Npcap modes (administrator-only access, 802.11 monitor mode, WinPcap compatibility) were not needed. No reboot was required: the interface list immediately showed **VMware Network Adapter VMnet8** with live traffic.

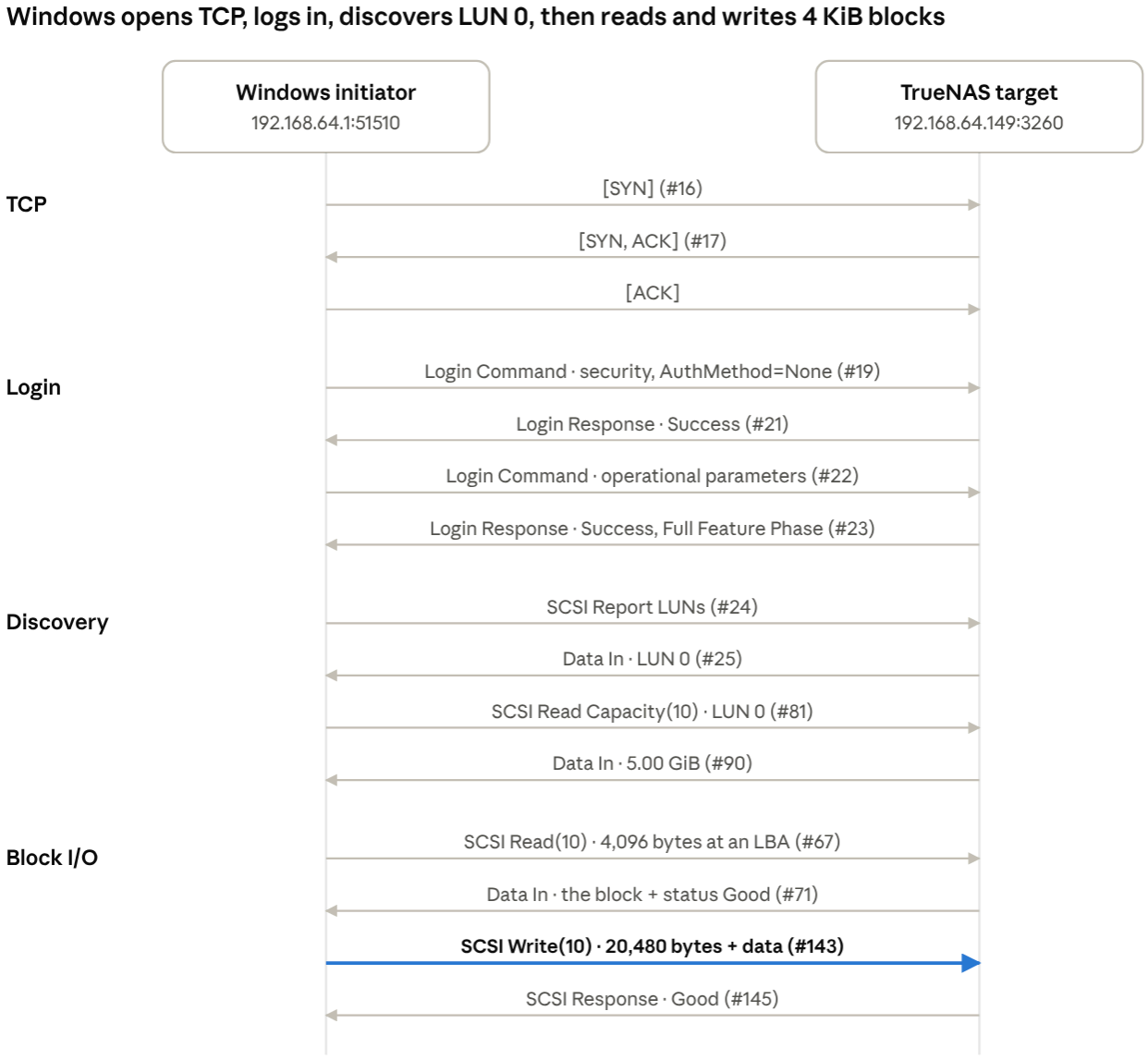
Image — Figure 54: Wireshark interface list, VMware Network Adapter VMnet8 with live tr…

A **capture filter** `tcp port 3260` was set before starting, so that only iSCSI traffic was recorded, not the web UI or the SMB traffic. A capture filter decides what is recorded; the display filters used later only decide what is shown.

Image — Figure 55: Capture filter tcp port 3260 accepted (green)

The procedure was: start the capture, disconnect and reconnect the target in the iSCSI Initiator, initialise and format the disk, write and read files on `I:`, stop the capture, and save it as `capture-iscsi.pcapng`.

9.2 The iSCSI session at a glance



iSCSI session reconstructed from the capture · Wireshark frame numbers in brackets

The diagram is reconstructed from the captured frames, numbered as in Wireshark. Every exchange follows the same request/response pattern: the initiator sends a command, the target answers with data and a status.

9.3 Display filters used

Filter	Shows	Result in this capture
`iscsi`	All iSCSI PDUs	NOP-In/Out keep-alives, Logout, Login, Report LUNs, SCSI commands

Filter	Shows	Result in this capture
`tcp.port == 3260 && tcp.flags.syn == 1`	Connection opening	Frames 16 [SYN] and 17 [SYN, ACK]
`iscsi.opcode == 0x03 || iscsi.opcode == 0x23`	Login Command / Login Response	Frames 19, 21, 22, 23
`scsi_sbc.opcode == 0x28`	SCSI Read(10) commands	1,041 packets displayed
`scsi_sbc.opcode == 0x2a`	SCSI Write(10) commands	1,078 packets displayed

9.4 Session control: keep-alive, logout and login

Image — Figure 56: Display filter iscsi, NOP keep-alives, Logout, Login and Report LUNs

- **NOP-In / NOP-Out:** every 32 seconds, TrueNAS sends a NOP-In and Windows answers with a NOP-Out. This keep-alive checks that the session is still alive.
- **Logout Command (Close session) / Logout Response** (frames 10, 12): the *Disconnect* clicked in the initiator.
- **Login Command / Login Response** (frames 19 to 23): the new *Connect*.
- **SCSI Report LUNs** (frame 24): Windows asks which LUNs the target offers; the answer is LUN 0.

9.5 The TCP connection

Image — Figure 57: TCP handshake, 51510 → 3260 [SYN] then 3260 → 51510 [SYN, ACK]

Windows opens the connection from an ephemeral port (51510) to the well-known iSCSI port **3260**: [SYN] in frame 16, [SYN, ACK] from TrueNAS in frame 17, then [ACK]. Both sides announce an **MSS of 1460 bytes**, the largest TCP payload per Ethernet frame. This is why large reads and writes later appear as many 1460-byte segments.

9.6 The Initiator/Target login

Image — Figure 58: Login exchange, two Login Command / Login Response pairs, both Succe…

The login text is readable in the hex dump of frame 19:

```
InitiatorName=iqn.1991-05.com.microsoft:<laptop name>
SessionType=Normal
TargetName=iqn.2005-10.org.freenas.ctl:iscsi-target
AuthMethod=None
```

The login runs in two stages:

1. **Security negotiation** (frames 19 and 21): the parties agree on authentication. `AuthMethod=None` matches the target configuration without CHAP.
2. **Operational negotiation** (frames 22 and 23): they agree on session parameters such as maximum data segment size and immediate data. The final *Login Response (Success)* moves the session to the **Full Feature Phase**, where SCSI commands may flow.

The login text travels in clear. Without CHAP or IPsec, anyone on the network could read the IQNs, which is why production SANs use CHAP and an isolated storage network.

9.7 Disk discovery and formatting

Right after login, Windows sends **SCSI Read Capacity(10) LUN 0** and TrueNAS answers **5.00 GiB**, the size of the zvol. Windows then reads the first blocks to look for a partition table.

Image — Figure 59: Read Capacity(10) on LUN 0 answered with 5.00 GiB

During initialisation and formatting (with BitLocker), the capture grew to 64,674 packets. Several command types are visible:

Image — Figure 60: Formatting traffic, Write(10), Synchronize Cache(10) and Unmap

Command	Meaning
SCSI Write(10) LUN 0x00, n bytes at LBA x	Write n bytes starting at logical block address x
SCSI Response (Good)	TrueNAS confirms the command succeeded
SCSI Synchronize Cache(10)	Windows asks TrueNAS to flush its write cache to stable storage

Command	Meaning
SCSI Unmap	TRIM: Windows reports blocks as free so ZFS can release them

9.8 A READ operation

Image — Figure 61: Display filter scsi_sbc.opcode == 0x28, Read(10) and its Data In

- **Frame 67, Windows → TrueNAS:** `SCSI Read(10) LUN: 0x00 4096 bytes (1 blocks) at LBA …`. Windows asks for one 4 KiB block at a precise address. The iSCSI flags `0xc1, F, R` mean Final PDU and Read.
- **Frame 71, TrueNAS → Windows:** `SCSI Data In (Good)`. The block's data and the status Good arrive together.

In a read, data flows from the target to the initiator.

9.9 A WRITE operation

Image — Figure 62: Display filter scsi_sbc.opcode == 0x2a, Write(10) with its payload a…

- **Frame 143, Windows → TrueNAS:** `SCSI Write(10) LUN: 0x00 20480 bytes (5 blocks)`. The flags `0xa1, F, W` mean Final and Write. The line *15 Reassembled TCP Segments (20528 bytes)* shows a 48-byte iSCSI header followed by 20,480 bytes of data, carried inside the command (immediate data) and split across 15 TCP segments of up to 1,460 bytes. *SCSI Payload (Write(10) Request Data)* is the data itself.
- **Frame 145, TrueNAS → Windows:** `SCSI Response (Good)`. The blocks are stored.

In a write, data flows from the initiator to the target. Both directions end with a Good status. Because BitLocker encrypts the volume, the payload bytes are ciphertext; the SCSI commands, LUN, LBA and lengths remain readable.

9.10 Encapsulation chain

Expanding frame 143 in Wireshark shows each protocol carried inside the one below it:

Layer	Content in frame 143
SCSI	CDB Write(10): opcode 0x2A, logical block address, number of blocks, plus 20,480 bytes of data

Layer	Content in frame 143
iSCSI	PDU *SCSI Command*: 48-byte header with LUN 0, flags F and W, task tag
TCP	Source port 51510 → destination port 3260; reliable, ordered delivery in segments of up to 1,460 bytes
IP	192.168.64.1 → 192.168.64.149
Ethernet II	MAC 00:50:56:c0:00:08 (Windows VMnet8 adapter) → 00:0c:29:9a:5d:15 (TrueNAS)

iSCSI takes the SCSI commands a computer would send to a local disk over a cable and wraps them in TCP/IP, so they can cross an ordinary Ethernet network.

9.11 Capture quality

The status bar reports **2,516 dropped packets (3.7 %)**. They were lost during the burst of writes caused by BitLocker encryption, when Wireshark could not keep up. The packets needed for this analysis (handshake, login, discovery, reads and writes) were all captured.

10. Comparison of NFS, SMB and iSCSI

NFS and SMB share files and keep the file system on the server; iSCSI shares a disk and leaves the file system to the client. The table compares the three protocols as configured and observed in this lab.

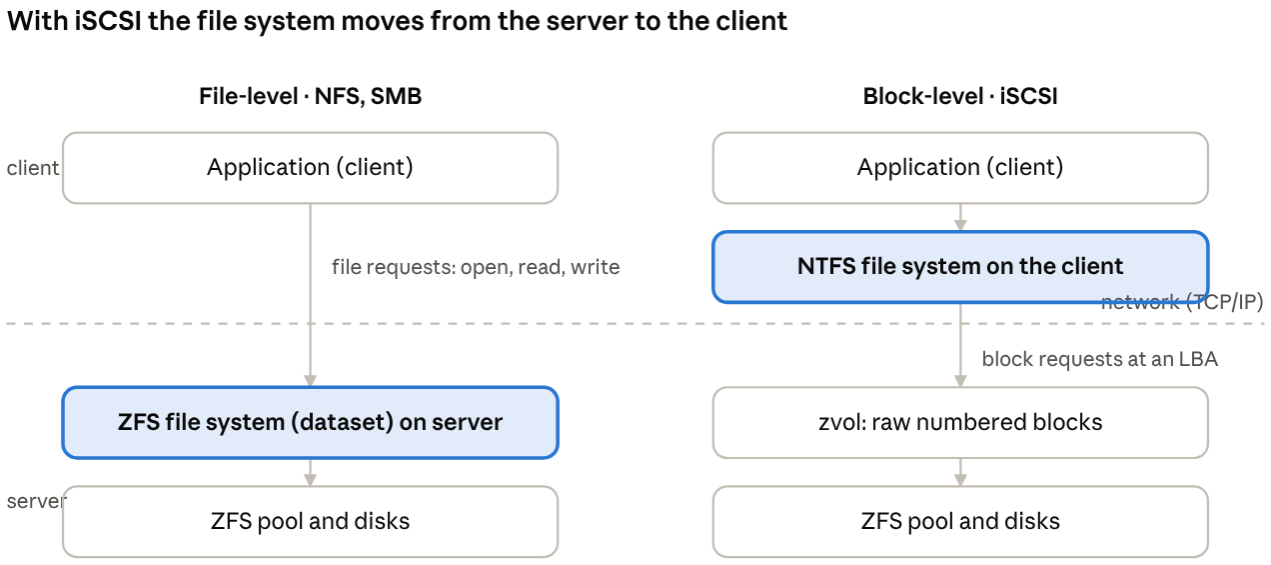
Criterion	NFS	SMB	iSCSI
Storage type	File-level	File-level	Block-level
Typical clients	Linux, UNIX	Windows (also macOS, Linux)	Any OS with an initiator, hypervisors, databases
ZFS object shared	Dataset `nfs-share`	Dataset `smb-share`	Zvol `zvol-iscsi`
Transport and port	TCP 2049 (NFSv4.2)	TCP 445	TCP 3260
Who manages the file system	TrueNAS (ZFS)	TrueNAS (ZFS, through Samba)	The client (NTFS on Windows)

Criterion	NFS	SMB	iSCSI
How it appears on the client	Directory mounted on `/mnt/nfs`	Network folder `\\192.168.64.149\smb-share`	Local disk `I:` in This PC
Authentication in this lab	None; trust by IP network and UID/GID (`sec=sys`)	User and password (`smbuser`)	None (CHAP possible)
File ownership on the server	root (Mapall)	smbuser	Not visible: server sees only blocks
Concurrent access by several clients	Yes, designed for it	Yes, with file locking	No, unless a cluster file system is used
Can TrueNAS see the files?	Yes	Yes	No
Typical use	Shared home directories, Linux servers, containers	Office file shares, Windows networks	Virtual machine disks, databases, boot disks

In short: choose NFS or SMB when several users or machines must share the same files, and iSCSI when one machine needs a fast disk it can format and control itself.

11. File-level vs block-level storage

The difference between the two models is where the file system lives. In file-level storage it lives on the server; in block-level storage it lives on the client.



Where the file system lives in each model

File-level storage (NAS: NFS, SMB). The client asks for files by name: open `test.txt`, append a line, delete `copie.txt`. The server translates these requests into operations on its own file system (here a ZFS dataset). The server knows every file, its owner and its permissions, so several clients can safely share the same data. The lab proved this when `ls` on TrueNAS listed exactly the files created from Ubuntu and Windows.

Block-level storage (SAN: iSCSI). The client asks for numbered blocks: read 1 block of 4,096 bytes at a given logical block address, write 5 blocks at another. The server stores those blocks in the zvol without knowing what they contain. The client builds its own file system on top (Windows created GPT and NTFS, then BitLocker). The lab proved this when `ls /mnt/pool-iscsi` on TrueNAS was empty while Windows showed two files on `I:`.

	File-level	Block-level
Unit of access	File, directory	Block (logical block address)
File system location	Server	Client
Sharing between clients	Native	Needs a cluster-aware file system
Performance overhead	Extra file-system layer on the server	Lower; behaves like a local disk
Server visibility of data	Full (names, owners, content)	None (only blocks, here encrypted)
Lab example	`nfs-share`, `smb-share`	`zvol-iscsi` as drive `I:`

12. Problems encountered and solutions

Six problems came up during the lab; all were solved without losing data. The most significant was the duplicate-serial error, which blocked pool creation in the web UI.

#	Problem	Cause	Solution
1	VM files created inside OneDrive (blue sync arrows in Explorer)	VMware's default folder was the synced Documents folder	VM removed from the library, folder moved to `C:\VMs\TrueNAS`, reopened with *I Moved It*
2	VM would boot in legacy BIOS	No `firmware` line in the `.vmx` file	Added `firmware = "efi"`; the installed GRUB then showed *UEFI Firmware Settings*
3	Pool creation refused: *Disks have duplicate serial numbers: None (sda, sdb, sdc)*	VMware's emulated SCSI controller returned an empty serial for each disk; TrueNAS blocks pools on disks it cannot tell apart, to avoid building RAID on one disk seen twice (multipath)	Changing the controller to PVSCSI did not help. Pools created through the API with `midclt call -j pool.create … "allow_duplicate_serials": true`, safe because the disks are truly separate

#	Problem	Cause	Solution
4	Pasted command failed with `zsh: bad pattern: ^[[200~`	The browser shell passed bracketed-paste markers to zsh	`unset zle_bracketed_paste`, then paste again
5	`midclt: error: unrecognized arguments: -ob`	Wrong flag: the job option is `-j`, so `-job` was read as `-j` plus `ob`	Used `-j`
6	First Wireshark attempt missed the iSCSI login	The initiator connected before the capture was started	Started the capture, then disconnected and reconnected the target

Smaller observations: the Windows file was saved as `test-smb.txt.txt` because Explorer hides known extensions; TrueNAS uses US/Pacific time by default, so its timestamps were eight hours behind Tunis time; and the web UI session expires after inactivity, requiring a new login as `truenas_admin`.

13. Answers to the synthesis questions

Q1. What is the difference between RAID, VDEV, Pool, Dataset and zvol?

RAID is the technique of combining several disks for redundancy or performance (mirroring, parity). In ZFS, RAID is applied inside a **VDEV**, a group of disks with one layout: `mirror-0` (sdb + sdc) or `raidz1-0` (sdd + sde + sdf). A **pool** is the storage space built from one or more VDEVs; ZFS stripes data across them and the pool is the unit of capacity (`pool-nfs`, 9.2 GiB). A **dataset** is a file system created inside a pool, with files, permissions, compression and quotas; it is what NFS and SMB share. A **zvol** is a block device created inside a pool, a fixed-size range of blocks with no file system; it is what iSCSI shares.

Q2. Why do NFS and SMB provide file-level storage?

Because the client accesses named files and folders, and the server manages the file system. The client sends operations such as open, read, write and delete on a path; TrueNAS performs them on its ZFS dataset and keeps ownership and permissions. The lab showed this: files created from Ubuntu and Windows were directly visible with `ls` on the server, owned by `root` (NFS Mapall) and by `smbuser` (SMB).

Q3. Why does iSCSI provide block-level storage?

Because the client accesses numbered blocks of a raw disk, not files. iSCSI carries SCSI commands such as Read(10) and Write(10) that name a logical block address and a number of blocks. TrueNAS exposes a zvol that it does not interpret; Windows created the partition table and the NTFS file system itself. On the server, `/mnt/pool-iscsi` stayed empty and the zvol appeared only as the block device `/dev/zd0`.

Q4. What is the role of the LUN in iSCSI?

A LUN (Logical Unit Number) identifies one logical disk behind a target. A single target can present several LUNs, and the initiator addresses each SCSI command to a specific one. In this lab, the extent `zvol-iscsi` was associated with the target as **LUN 0**. Windows discovered it with *Report LUNs*, measured it with *Read Capacity(10) LUN 0* (5.00 GiB), and every Read(10) and Write(10) in the capture carries `LUN: 0x00`.

Q5. What is the role of the Target and the Initiator?

The **initiator** is the client: it starts the session and sends SCSI commands. Here it is the Microsoft iSCSI Initiator, named `iqn.1991-05.com.microsoft:…`. The **target** is the server endpoint that receives the commands and executes them on its storage. Here it is `iqn.2005-10.org.freenas.ctl:iscsi-target`, listening on portal 0.0.0.0:3260 and exposing LUN 0. The initiator logs in to the target, both negotiate security and parameters, then the initiator sends commands and the target returns data and status.

Q6. How is a SCSI command carried by iSCSI/TCP?

The SCSI command (a CDB such as Write(10), opcode 0x2A, with the address and length) is placed in an **iSCSI PDU** of type *SCSI Command*, with a 48-byte header containing the LUN, the flags (F, W) and a task tag; write data can follow in the same PDU. The PDU is sent as a byte stream over a **TCP** connection to port 3260, which splits it into segments of up to 1,460 bytes and guarantees order and delivery. Each segment is placed in an **IP** packet (192.168.64.1 → 192.168.64.149), and each packet in an **Ethernet** frame (MAC 00:50:56:c0:00:08 → 00:0c:29:9a:5d:15). In frame 143, a 20,528-byte iSCSI PDU (48-byte header plus 20,480 bytes of data) travelled in 15 reassembled TCP segments. The target reverses the process and returns a *SCSI Response*, or *Data In* for a read.

Q7. What can be identified in Wireshark during a READ and a WRITE?

For a **READ** (frames 67 and 71): the SCSI command *Read(10)*, the LUN (0x00), the logical block address, the requested length (4,096 bytes, 1 block), the iSCSI flags F and R, the direction (initiator → target for the command, target → initiator for the data), and the *Data In* PDU returning the block with status Good.

For a **WRITE** (frames 143 and 145): the command *Write(10)*, the LUN, the address, the length (20,480 bytes, 5 blocks), the flags F and W, the data payload carried with the command and split over 15 TCP segments, and the *SCSI Response (Good)* confirming the write.

In both cases the full encapsulation is visible (Ethernet, IP, TCP port 3260, iSCSI, SCSI), along with sequence numbers and timing. Here the payload bytes were encrypted by BitLocker, so the content of the files could not be read.

14. Conclusion and possible improvements

The mini-project achieved every objective: one TrueNAS server provided mirrored and RAIDZ1 ZFS pools, served files to Linux over NFS and to Windows over SMB, and served a raw disk to Windows over iSCSI, with the iSCSI exchanges captured and explained in Wireshark.

The main lesson is the position of the file system. With NFS and SMB, the server owns the file system and shares files, which makes multi-user sharing simple. With iSCSI, the server only stores blocks and the client owns the file system, which gives the performance and control of a local disk. The lab made this concrete: the same `ls` command listed every NFS and SMB file on the server, but nothing at all for iSCSI.

The lab also showed that real deployments need care beyond configuration screens: unique disk identifiers for ZFS, correct timing for packet captures, and attention to small details such as hidden file extensions.

Possible improvements for a production setup:

- **Security:** use HTTPS for the web UI, enable CHAP authentication on the iSCSI target, restrict initiators by IQN, and replace NFS Mapall with matching users or Kerberos (`sec=krb5`).
- **Network:** put storage traffic on a dedicated network or VLAN, separate from user traffic.
- **Resilience:** use RAIDZ2 or mirrors with more disks, add a hot spare, and schedule periodic scrubs and SMART tests.
- **Data protection:** configure periodic ZFS snapshots and replication to a second system.
- **Hardware:** on real servers, use disks with genuine serial numbers so the duplicate-serial safety check never needs to be bypassed.

Appendix: command reference

TrueNAS shell (System > Shell):

```
unset zle_bracketed_paste
sudo midclt call -j pool.create '{"name": "pool-nfs", "topology": {"data":
[{"type": "MIRROR", "disks": ["sdb", "sdc"]}]}, "allow_duplicate_serials":
true}'
ls -l /mnt/pool-nfs/nfs-share
ls -l /mnt/pool-smb/smb-share
ls -la /mnt/pool-iscsi
ls -l /dev/zvol/pool-iscsi/
```

Ubuntu NFS client:

```
sudo apt update && sudo apt install -y nfs-common
showmount -e 192.168.64.149
sudo mkdir -p /mnt/nfs
sudo mount -t nfs 192.168.64.149:/mnt/pool-nfs/nfs-share /mnt/nfs
df -h /mnt/nfs
mount | grep nfs
```

Windows: `\\192.168.64.149\smb-share` in File Explorer; iSCSI Initiator Quick Connect to `192.168.64.149`; `diskmgmt.msc` for Disk Management.

Wireshark: capture filter `tcp port 3260`; display filters `iscsi`, `tcp.port == 3260 && tcp.flags.syn == 1`, `iscsi.opcode == 0x03 || iscsi.opcode == 0x23`, `scsi_sbc.opcode == 0x28` (Read(10)), `scsi_sbc.opcode == 0x2a` (Write(10)). The capture is saved as `capture-iscsi.pcapng`.